Test Plan

Supply Chain Management Simulator

Field	Value
Project	Supply Chain Management Simulator
Version	1.1
Authors	Team 02 – Rohith Karning, Rohanth Ganesha C, Rishi J Mule, Pranav Achar
Date	09-11-2026
Status	Final (pending approval)

Revision History

Version	Date	Author(s)	Description of change	Approval
1.1	09-11-2026	Team 02	Updated for Sprint 1 and 2 changes and final documentation audit (see `docs/deviations.md`)	Pending
1.0	11-10-2026	Team 02	Initial draft	Pending

Table of Contents

1. Introduction
2. Test Items
3. Features to Be Tested
4. Features Not to Be Tested
5. Test Approach
    - 5.1 Security Validation
6. Test Environment
7. Test Schedule
8. Test Deliverables
9. Roles and Responsibilities
10. Risks and Mitigation
11. Assumptions and Dependencies
12. Suspension and Resumption Criteria
13. Test Case Management and Traceability
14. Test Metrics and Reporting
15. Approvals

Appendix: Test Cases (TC-INV-01, TC-INV-02, TC-SIM-01, TC-SIM-02, TC-ORD-01, TC-ORD-02, TC-PERF-01, TC-COV-01, TC-SHP-01, TC-SHP-02, TC-SEC-01, TC-SEC-02, TC-ANA-01, TC-ANA-02, TC-REP-01)

---

1. Introduction

1.1 Purpose

This Test Plan describes how the **Supply Chain Management Simulator** (Python package `scm_sim`), version 1.1, is verified against its Software Requirements Specification. It defines what is tested and what is not, the test levels and techniques used, the environment, the schedule, the people responsible, the entry, exit, suspension and resumption criteria, and how results are recorded and traced back to requirements.

The plan has two goals:

1. To show that every functional requirement (SCM-F-001 to SCM-F-044), non-functional requirement (SCM-NF-001 to SCM-NF-005) and security requirement (SCM-SR-001 to SCM-SR-007) in the SRS is covered by at least one test case.
2. To give evaluators and team members a repeatable procedure: each automated test case names the pytest tests that implement it, so anyone can re-run them with `python -m pytest -v --cov=scm_sim --cov-branch`.

1.2 Scope

In scope. The plan covers the seven test items listed in §2: the Inventory, Order Manager, Shipment Manager, Analytics, Simulation Engine and CLI components, and the CI Pipeline. The simulator is installable as a package and exposes both `scm-sim` and `python -m scm_sim`. It includes:

- **Unit testing** of each module through its own test file in `tests/`.
- **Integration testing** of the modules working together in the Simulation Engine (`tests/test_simulator.py`).
- **System testing** of the command-line interface, both by calling `main()` directly and by running `python -m scm_sim` in a subprocess.
- **Acceptance testing** of the demo scenario against the problem statement: goods move from suppliers to warehouses to customers, and delays, stockouts and excess inventory are detected.
- **Non-functional testing**: performance (SCM-NF-001), reproducibility (SCM-NF-002), coverage and maintainability (SCM-NF-003), portability (SCM-NF-004) and usability (SCM-NF-005).
- **Security validation** of data-integrity requirements (input validation, invariants, guarded status changes, unknown-ID rejection, copies of returned data) and of the development process (branch protection, no secrets, no runtime dependencies).

The plan contains **15 test cases**. Eleven are fully automated and run in the CI `test` job. TC-PERF-01 and TC-SEC-02 are manual, and TC-COV-01 and TC-REP-01 combine automated checks with a manual part. The final unit-test report records 100 automated tests passed; the combined line and branch coverage evidence is 99.66%. TC-COV-01 remains partially validated because macOS was not tested.

Out of scope. The features listed in §4 are not tested, including the Python interpreter and standard library, pytest and GitHub Actions themselves, and functions the system does not provide (graphical or web interfaces, storage, networking).

1.3 References

ID	Document	Version / Date
[SRS]	Software Requirements Specification: Supply Chain Management Simulator	1.1, 09-11-2026
[SAD]	Software Architecture Document: Supply Chain Management Simulator	1.1, 09-11-2026
[REG]	Requirements registry, `docs/requirements-`	11-10-2026

ID	Document	Version / Date
	`registry.md`	
[IEEE829]	IEEE Std 829-2008, Standard for Software and System Test Documentation	2008
[IEEE29119]	ISO/IEC/IEEE 29119-3, Software Testing: Test Documentation	2021
[REPO]	Team repository, https://github.com/rohithkarning-debug/supply-chain-simulator (`README.md`, `tests/`, `.github/workflows/ci.yml`)	Final Sprint 2 implementation; see dated evidence reports in `docs/test-reports/`
[PYTEST]	pytest documentation, https://docs.pytest.org	pytest ≥ 8.0
[PYCOV]	pytest-cov documentation, https://pytest-cov.readthedocs.io	pytest-cov ≥ 5.0

1.4 Definitions, Acronyms and Abbreviations

Term	Definition
Test item	A software component (or the CI workflow) that is the object of testing
Test case (TC)	A set of preconditions, inputs, steps and expected results for verifying one or more requirements, identified as `TC-<area>-NN`
Automated test	A pytest function in `tests/` that checks expected results with `assert` and runs without human action
Manual test	A test executed and recorded by a person following the written steps
Unit / Integration / System / Acceptance test	Testing one module on its own / modules working together / the whole program through its CLI / the demo run against the problem statement
Boundary-value testing	Testing at and just beyond a limit, e.g. stock levels 20 and 21 for a reorder point of 20
Regression testing	Re-running the whole suite after every change to detect newly broken behaviour; done automatically by CI on every pull request
Coverage	Line coverage and branch coverage of `scm_sim`, measured by pytest-cov; the CI gate measures combined line and branch coverage
CI run	One execution of the GitHub Actions workflow `CI` (job `test`)
Entry / Exit criteria	Conditions that must hold before testing starts / before testing is considered complete
Defect	A difference between actual and expected results, recorded as a GitHub Issue
Pass / Fail	The actual result matches / does not match the expected result in every step
Not run	A manual test case that has not yet been executed
SKU, Stockout, Back-order, Fill rate, Seed	As defined in [SRS] §1.4
SRS / SAD / RTM	Software Requirements Specification / Software

Term	Definition
	Architecture Document / Requirements Traceability Matrix
CI / CLI / PR	Continuous Integration / Command-Line Interface / Pull Request

2. Test Items

2.1 Basis of Testing

The items subject to test are the seven components of the Supply Chain Management Simulator as built and merged into `main` at commit **`b3702ef`** (11-10-2026). The system is a Python 3.11 command-line application whose source lives in the `src/scm_sim/` package, whose automated tests live in `tests/`, and whose continuous verification runs as the `test` job of the GitHub Actions workflow `.github/workflows/ci.yml`. Every item below is a module or a repository-level process artefact that is exercised by at least one planned test case in Test Plan §13.

At that commit the baseline is **73 automated tests passing with 97% line coverage of `scm_sim`**, CI green. These figures are recorded here so that the later test report can be compared against a fixed baseline rather than against an expectation formed after the fact.

2.2 Test Items

#	Test item	Source file	Owner	Responsibility under test	Requirements	Test cases
T1	**Inventory**	`src/scm_sim/inventory.py`	Rohith Karning	Stock levels per (warehouse, SKU); validated `add` and `remove`; registered warehouse capacities and the `CapacityExceededError` invariant; per-item `StockPolicy` with `needs_reorder` and `is_excess`; sorted reorder suggestions; independent stock snapshots.	SCM-F-001, SCM-F-002, SCM-F-003, SCM-F-004; SCM-SR-002, SCM-SR-007	TC-INV-01, TC-INV-02, TC-SEC-01
T2	**Order Manager**	`src/scm_sim/orders.py`	Rohanth Ganesha C	Customer order creation with sequential `ORD-0001`…	SCM-F-010, SCM-F-011, SCM-F-012, SCM-F-013; SCM-SR-003,	TC-ORD-01, TC-ORD-02, TC-SEC-01

#	Test item	Source file	Owner	Responsibility under test	Requirements	Test cases
				identifiers; all-or-nothing fulfilment; back-ordering when stock is short; oldest-first retry of open orders; cancellation, lookup by ID and listing by status.	SCM-SR-004	
T3	**Shipment Manager**	`src/scm_sim/shipments.py`	Rishi J Mule	Supplier registration with lead time and reliability; dispatch with sequential `SHP-0001`… identifiers; stochastic delay modelling from the seeded PRNG; arrival processing into inventory; listing of in-transit and late shipments.	SCM-F-020, SCM-F-021, SCM-F-022, SCM-F-023; SCM-SR-001, SCM-SR-004	TC-SHP-01, TC-SHP-02, TC-SEC-01
T4	**Analytics**	`src/scm_sim/analytics.py`	Pranav Achar	Detection of delayed shipments with days late, ordered worst first; per-item stock alerts with fallback thresholds; order metrics and fill rate; daily report composition/ formatting; and full-run summary composition/	SCM-F-030, SCM-F-031, SCM-F-032, SCM-F-033, SCM-F-034; SCM-NF-002, SCM-NF-005	TC-ANA-01, TC-ANA-02

#	Test item	Source file	Owner	Responsibility under test	Requirements	Test cases
				formatting.		
T5	**Simulation Engine**	`src/scm_sim/simulator.py`	Rohith Karning	Scenario configuration; fixed-order daily time-step (arrivals → back-order retry → today's orders → replenishment → report); retention of every daily report; automatic capacity-trimmed replenishment; built-in demo scenario; seeded reproducibility; and one flattened metrics row per day for CSV export.	SCM-F-040, SCM-F-041, SCM-F-042, SCM-F-044; SCM-NF-001, SCM-NF-002	TC-SIM-01, TC-SIM-02, TC-REP-01, TC-PERF-01
T6	**CLI**	`src/scm_sim/__main__.py`	Rohith Karning	Installable `scm-sim` and `python -m scm_sim` entry points; day-by-day log and final report; exit code 0 on success; usage error with exit code 2 for invalid `--days`; `--help`; optional `--summary` display and `--csv PATH` export.	SCM-F-034, SCM-F-043, SCM-F-044; SCM-NF-005	TC-SIM-02, TC-PERF-01
T7	**CI Pipeline**	`.github/workflows/`	Rohith Karning	Execution of the whole	SCM-NF-003; SCM-SR-005,	TC-COV-01, TC-SEC-02

#	Test item	Source file	Owner	Responsibility under test	Requirements	Test cases
		`ci.yml`		suite with `pytest -v --cov=scm_sim --cov-branch --cov-fail-under=90` on pushes to `main` and pull requests; publication of the required `test` check; enforcement of the protected-`main` rules that make merging conditional on it.	SCM-SR-006	

2.3 Items Not Under Test

The following are explicitly excluded from the test items, with the reasons recorded in Test Plan §4: the Python interpreter and standard-library internals (including the statistical behaviour of `random`, which is trusted rather than verified); the internals of pytest, pytest-cov and GitHub Actions; and any GUI, web interface, database or authentication layer, none of which exist in this system. The test baseline for all items is commit `b3702ef`; any change to that baseline requires the affected items to be re-tested.

2.4 Test Item Interfaces

The items are verified through their public interfaces, which are documented in SAD §4.3. The interfaces below are the contract each test case relies on; a change to any of them invalidates the corresponding test case and requires it to be updated in the same change.

Test item	Interfaces exercised by the test cases
T1 Inventory	`Inventory()`, `level`, `add`, `remove`, `register_warehouse`, `total_in_warehouse`, `set_policy`, `get_policy`, `needs_reorder`, `is_excess`, `reorder_suggestions`, `snapshot`, `StockPolicy`
T2 Order Manager	`OrderManager(inventory)`, `create`, `get`, `try_fulfil`, `process_open`, `cancel`, `by_status`, `OrderStatus`
T3 Shipment Manager	`Supplier(...)`, `ShipmentManager(inventory, rng)`, `add_supplier`, `dispatch`, `process_arrivals`, `in_transit`, `delayed`, `ShipmentStatus`, `Shipment.days_late`
T4 Analytics	`find_delayed_shipments`, `find_stockouts`, `find_low_stock`, `find_excess`, `order_metrics`, `build_report`,

Test item	Interfaces exercised by the test cases
	`format_report`, `summarize_run`, `format_run_summary`
T5 Simulation Engine	`Simulation(seed, reorder_point, max_level)`, `add_supplier`, `add_warehouse`, `set_policy`, `add_stock`, `schedule_order`, `step`, `run`, `final_summary`, `metrics_rows`, `build_demo`
T6 CLI	`scm-sim`, `python -m scm_sim`, `--days`, `--seed`, `--summary`, `--csv`, `--help`, process exit code
T7 CI Pipeline	GitHub Actions job `test` on `ubuntu-latest` with Python 3.11; GitHub branch protection settings for `main`

3. Features to Be Tested

This section enumerates every feature of the Supply Chain Management Simulator that is to be tested, mapped to the requirement identifiers defined in the Software Requirements Specification. It is the authoritative list of what this Test Plan covers: a requirement that appears in this section is in scope for verification, and a requirement that does not appear here is either out of scope (Test Plan §4) or an objective rather than a testable requirement.

The identifiers below are taken from the Requirements Registry (`docs/requirements-registry.md`) and match SRS §4, §5 and §5.1 exactly. Thirty-four requirements are in scope: 22 functional, 5 non-functional and 7 security requirements, supported by the two security objectives SCM-SO-1 and SCM-SO-2.

3.1 Feature Group A — Inventory Management

Tests the behaviour of the Inventory component (`src/scm_sim/inventory.py`), which owns all stock state.

Req ID	Feature to be tested	Verifiable behaviour	Component	Test level	Test case
SCM-F-001	Stock tracking and validated mutation	Quantity is tracked per (warehouse, SKU) pair; `add` and `remove` reject quantities ≤ 0 with `ValueError`; removing more than available raises `InsufficientStockError` and leaves the level unchanged; a never-stocked item reads as level 0.	Inventory	Unit	TC-INV-01
SCM-F-002	Warehouse capacity enforcement	`register_warehouse` rejects a capacity ≤ 0	Inventory	Unit	TC-INV-01

Req ID	Feature to be tested	Verifiable behaviour	Component	Test level	Test case
		with `ValueError`; an `add` that would push the warehouse total past its capacity raises `CapacityExceededError` and leaves stock unchanged; an `add` reaching exactly the capacity succeeds; an unregistered warehouse has unlimited capacity.			
SCM-F-003	Per-item stock policy	`StockPolicy` defaults to reorder point 20, reorder quantity 100 and max level 500; negative values and `max_level < reorder_point` are rejected with `ValueError`; `needs_reorder` is true when level ≤ reorder point; `is_excess` is true only when level > max level.	Inventory	Unit	TC-INV-02
SCM-F-004	Reorder suggestions and snapshot	`reorder_suggestions()` returns only tracked items at or below their reorder point, each with the policy's reorder quantity, in sorted order; `snapshot()` returns every level as an	Inventory	Unit	TC-INV-02

Req ID	Feature to be tested	Verifiable behaviour	Component	Test level	Test case
		independent copy that cannot alter internal state.			

3.2 Feature Group B — Order Management

Tests the behaviour of the Order Manager component (`src/scm_sim/orders.py`), which owns the customer order book.

Req ID	Feature to be tested	Verifiable behaviour	Component	Test level	Test case
SCM-F-010	Order creation and identifier sequence	`create` rejects a quantity ≤ 0 with `ValueError`; identifiers are allocated sequentially as `ORD-0001`, `ORD-0002`, …; a new order has status `PENDING` and records its creation day.	Order Manager	Unit	TC-ORD-01
SCM-F-011	Fulfilment and back-ordering	An order is fulfilled in full when stock $\geq$ quantity, with stock reduced, status `FULFILLED` and the fulfilment day recorded; otherwise it becomes `BACKORDERED` with stock unchanged; there is no partial fulfilment.	Order Manager	Unit	TC-ORD-01
SCM-F-012	Oldest-first retry of open orders	`process_open` retries all `PENDING` and `BACKORDERED` orders in order of creation day and then identifier, and returns only the orders newly	Order Manager	Unit	TC-ORD-02

Req ID	Feature to be tested	Verifiable behaviour	Component	Test level	Test case
		fulfilled on this pass.			
SCM-F-013	Cancellation, lookup and filtering	`cancel` rejects an already `FULFILLED` order with `ValueError` and an unknown identifier with `KeyError`; `get` raises `KeyError` for an unknown identifier; `by_status` returns exactly the orders in the requested status.	Order Manager	Unit	TC-ORD-02

3.3 Feature Group C — Shipment and Supplier Management

Tests the behaviour of the Shipment Manager component (`src/scm_sim/shipments.py`), which owns suppliers and inbound shipments.

Req ID	Feature to be tested	Verifiable behaviour	Component	Test level	Test case
SCM-F-020	Supplier registration and validation	`Supplier` rejects `lead_time_days < 0` and a `reliability` outside $[0.0, 1.0]$ with `ValueError`; `add_supplier` rejects a duplicate identifier with `ValueError` and an unknown identifier on dispatch with `KeyError`.	Shipment Manager	Unit	TC-SHP-01
SCM-F-021	Dispatch and identifier sequence	`dispatch` creates a shipment with status `IN_TRANSIT`, identifier `SHP-0001`, `SHP-`	Shipment Manager	Unit	TC-SHP-01

Req ID	Feature to be tested	Verifiable behaviour	Component	Test level	Test case
		`0002`, …, and `expected_day = day + supplier.lead_time_days`; a quantity ≤ 0 raises `ValueError`; `in_transit()` returns only shipments not yet delivered.			
SCM-F-022	Stochastic delay model	When no explicit delay is given, a PRNG draw exceeding the supplier's reliability adds `rng.randint(1, 3)` days to the arrival day and a draw within reliability adds none; `reliability = 1.0` is never delayed; an explicit `delay` overrides the random draw.	Shipment Manager	Unit	TC-SHP-02
SCM-F-023	Arrival processing and late detection	Every `IN_TRANSIT` shipment whose arrival day is reached is delivered, adding stock through `Inventory.add`, setting status `DELIVERED` and recording the actual day; a shipment is never delivered before its arrival day; `delayed(current_day)` returns shipments whose	Shipment Manager	Unit / Integration	TC-SHP-02, TC-SIM-01

Req ID	Feature to be tested	Verifiable behaviour	Component	Test level	Test case
		`days_late` exceeds 0, whether still in transit or already delivered late.			

3.4 Feature Group D — Analytics and Reporting

Tests the behaviour of the Analytics component (`src/scm_sim/analytics.py`), the read-only observation layer owned by the Dev Lead.

Req ID	Feature to be tested	Verifiable behaviour	Component	Test level	Test case
SCM-F-030	Delayed-shipment detection	`days_late` is the delivered shipment's actual day, or the current day while in transit, minus the expected day; only `days_late > 0` is reported; on-time and early arrivals are excluded; the result is ordered most-late first with a deterministic tie-break.	Analytics	Unit	TC-ANA-01
SCM-F-031	Inventory health classification	A stockout is a quantity of exactly 0; low stock is $0 < qty \leq$ reorder point (20 included, 0 and 21 excluded); excess is $qty >$ max level (500 excluded, 501 included) and reports the surplus; each class is returned sorted, and caller-supplied thresholds override the 20 / 500 defaults.	Analytics	Unit	TC-ANA-01

Req ID	Feature to be tested	Verifiable behaviour	Component	Test level	Test case
SCM-F-032	Order metrics and fill rate	Status counts for `PENDING`, `FULFILLED`, `BACKORDERED` and `CANCELLED` are correct and `total` equals the list length; fill rate is round(fulfilled ÷ (total − cancelled), 2); it is 0.0 when there are no eligible orders; enumeration members and plain strings are accepted, including mixed in one list.	Analytics	Unit	TC-ANA-02
SCM-F-033	Report composition and formatting	`build_report` returns a dictionary whose key set is exactly `{day, delayed_shipments, stockouts, low_stock, excess, orders}` and forwards caller thresholds; `format_report` emits the day header, one counted section per key, one line per item, and the text `No issues` for every empty section.	Analytics	Unit	TC-ANA-02
SCM-F-034	Run summary aggregation and display	`summarize_run` returns the count of days, the number of days with delays / stockouts / low stock / excess, the peak	Analytics / CLI	Unit / System	TC-ANA-02, TC-SIM-02

Req ID	Feature to be tested	Verifiable behaviour	Component	Test level	Test case
		delayed-shipments count, and the final fill rate; `format_run_summary` prints a single `=== Run Summary ===` block, displayed by the CLI when `--summary` is supplied.			

3.5 Feature Group E — Simulation Engine and Command-Line Interface

Tests the orchestration layer and its user-facing entry point.

Req ID	Feature to be tested	Verifiable behaviour	Component	Test level	Test case
SCM-F-040	Scenario configuration	Suppliers, warehouses with capacity, per-item policies linked to a supplier, initial stock and orders scheduled for future days can all be configured; a day in the past, a quantity ≤ 0 and an unknown warehouse are each rejected with the appropriate exception.	Simulation Engine	System	TC-SIM-02
SCM-F-041	Daily time-step and report retention	`step` advances the day counter and executes arrivals, back-order retry, today's orders, replenishment and report construction in that fixed order; `run(days)` returns one report per day	Simulation Engine	Integration	TC-SIM-01

Req ID	Feature to be tested	Verifiable behaviour	Component	Test level	Test case
		and `run(-1)` raises `ValueError`.			
SCM-F-042	Automatic replenishment	Exactly one replenishment is dispatched per item that needs reorder while nothing is in transit for it; the quantity is trimmed so that on-hand plus in-transit stock never exceeds the warehouse capacity.	Simulation Engine	Integration	TC-SIM-01
SCM-F-043	Command-line interface	`scm-sim` or `python -m scm_sim --days N --seed S` runs the demo, prints the day-by-day log and the final report, and exits with code 0; `--days` below 1 produces a usage error with exit code 2.	CLI	System	TC-SIM-02
SCM-F-044	Daily metrics CSV export	`--csv PATH` writes one metrics row per simulated day with the required header columns.	CLI	System	TC-SIM-02

3.6 Feature Group F — Non-Functional and Cross-Cutting Features

Req ID	Feature to be tested	Verifiable behaviour	Component	Test level	Test case
SCM-NF-001	Performance	A 365-day run of the demo scenario and the 10-day CLI demo including interpreter start-up each complete within 1 s.	Simulation Engine, CLI	Performance (manual)	TC-PERF-01
SCM-NF-002	Reproducibility	The same seed	Simulation	Reliability	TC-REP-01

Req ID	Feature to be tested	Verifiable behaviour	Component	Test level	Test case
		with the same scenario produces identical daily reports and identical output on every run and on every host.	Engine, Analytics	(automated + manual)	
SCM-NF-003	Coverage and pre-merge gate	The CI `test` job reports $\geq 90\%$ line coverage of `scm_sim` on every push to `main` and every pull request, and all tests pass before a merge is permitted.	CI Pipeline	Maintainability (automated)	TC-COV-01
SCM-NF-004	Portability and dependency constraint	The simulator runs on Python $\geq$ 3.11 on Windows, Linux and macOS using only the standard library at runtime, with third-party packages required for testing alone.	All modules, `requirements.txt`	Portability (automated + manual)	TC-COV-01
SCM-NF-005	Usability of the demo	A new user runs the demo with one documented command; every report section shows a count and prints "No issues" when empty; `--help` lists all options.	CLI, Analytics	System	TC-SIM-02

3.7 Feature Group G — Security

The two security objectives are not themselves testable; the features tested are the security requirements through which they are achieved.

Req ID	Feature to be tested	Verifiable behaviour	Component	Test level	Test case
SCM-SO-1	*Objective:*	Achieved	Inventory,	Security	TC-SEC-01

Req ID	Feature to be tested	Verifiable behaviour	Component	Test level	Test case
	integrity of simulation data	through SCM-SR-001 to SCM-SR-004 and SCM-SR-007 below; verified by TC-SEC-01 across all domain modules.	Order Manager, Shipment Manager		
SCM-SO-2	*Objective:* integrity and traceability of the codebase	Achieved through SCM-SR-005 and SCM-SR-006 below; verified by TC-SEC-02 against the repository and its branch-protection settings.	CI Pipeline, repository	Security (process)	TC-SEC-02
SCM-SR-001	Input validation before state change	Every public operation rejects invalid arguments — quantities ≤ 0, lead time < 0, reliability outside $[0,1]$, invalid policies, capacity ≤ 0, past days — with `ValueError` raised **before** any state changes.	All domain modules, Simulation	Security	TC-SEC-01
SCM-SR-002	Stock integrity invariants	Stock never falls below 0 or rises above a registered warehouse capacity; a violating operation raises `InsufficientStockError` or `CapacityExceededError` and leaves state unchanged.	Inventory	Security	TC-SEC-01
SCM-SR-003	Guarded order status	A `FULFILLED` order cannot be	Order Manager	Security	TC-SEC-01

Req ID	Feature to be tested	Verifiable behaviour	Component	Test level	Test case
	transitions	cancelled; `FULFILLED` and `CANCELLED` orders are never re-processed by a retry pass.			
SCM-SR-004	Rejection of unknown references	A reference to an unknown supplier, warehouse or order raises `KeyError`, and nothing is created implicitly as a side effect.	Order Manager, Shipment Manager, Simulation	Security	TC-SEC-01
SCM-SR-005	Protected `main` branch	A pull request is required to change `main`; at least one approval from a non-author is needed; the required `test` check must pass; the branch must be up to date; force pushes and deletions are blocked; the rules apply to administrators.	Repository, CI Pipeline	Security (process)	TC-SEC-02
SCM-SR-006	No secrets and no runtime dependencies	The repository contains no secrets or credentials; the simulator makes no network calls and has no runtime third-party dependency, with `requirements.txt` holding test-only packages.	All modules, repository	Security (process)	TC-SEC-02
SCM-SR-007	Returned data is a copy	Data returned to callers — snapshots and reports — is an	Inventory, Analytics	Security	TC-SEC-01

Req ID	Feature to be tested	Verifiable behaviour	Component	Test level	Test case
		independent copy, so modifying it cannot change internal state.			

3.8 Summary of Feature Coverage

Feature group	Functional	Non-functional	Security	Total
A — Inventory Management	4	0	0	4
B — Order Management	4	0	0	4
C — Shipment and Supplier Management	4	0	0	4
D — Analytics and Reporting	4	0	0	4
E — Simulation Engine and CLI	4	0	0	4
F — Non-Functional and Cross-Cutting	0	5	0	5
G — Security	0	0	7 requirements (+ 2 objectives)	7
Total	**20**	**5**	**7**	**32**

All 32 requirements of the SRS are assigned to a feature group and to at least one test case. The 15 test cases of Test Plan §13 therefore between them cover every requirement in the registry; none is left unverified and none is verified without a requirement to justify it.

4. Features Not to Be Tested

To optimize engineering resources and maintain strict alignment with the system boundary, the following aspects are intentionally excluded from testing:

Excluded Area	Justification / Boundary Rationale
Python 3.11 Runtime & Standard Library Internals	Internal mechanisms of Python core data structures (e.g., hash collision handling in `dict`, memory allocation in `dataclasses`, or internal parsing of `argparse`) are verified by CPython's upstream test suite. The project assumes standard library primitives operate according to specification.
Testing & CI Infrastructure Internals	The internal test discovery algorithms of `pytest`, coverage collection engines of `pytest-cov`, and virtual runner virtualization inside GitHub Actions (`ubuntu-latest`) are assumed correct. Verification focuses on our test assertions and code, not testing the test framework itself.
GUI, Web Interfaces, and Network Stacks	The simulator is strictly a local command-line and programmatic library. There are no web endpoints, HTTP servers, browser interfaces, REST APIs, or socket listeners implemented. Testing non-existent network interfaces is out of scope.
Persistent Databases & Storage Systems	All simulation state resides in volatile memory. There are no SQL databases, NoSQL engines, file-based persistence mechanisms, or ORM mappers.
Statistical Distribution of `random.Random`	The mathematical uniform distribution properties of

Excluded Area	Justification / Boundary Rationale
	the Mersenne Twister RNG (`random.Random`) are assumed correct as provided by the language runtime. Verification validates that the seed controls dispatch determinism and delay boundaries ($1 \leq \text{delay} \leq 3$), not statistical randomness across millions of draws.
Out-of-Scope Domain Features (SRS §1.2)	Commercial supply chain capabilities—such as real-time GPS fleet tracking, automated purchase order EDI integrations, multi-currency financial accounting, labor scheduling, and multi-echelon DC-to-store routing—are explicitly excluded from the educational project scope.

5. Test Approach

The testing strategy establishes a multi-tiered verification framework designed to guarantee domain correctness, state invariant enforcement, and regression protection across all development phases.

5.2 Test Levels

1. **Unit Testing:** Conducted independently for each isolated domain module (`test_inventory.py`, `test_orders.py`, `test_shipments.py`, and `test_analytics.py`). Focuses on individual class methods, function inputs/outputs, boundary edge cases, and exception triggers.
2. **Integration Testing:** Conducted within `tests/test_simulator.py`. Validates the interaction contracts between domain components when orchestrated by `Simulation.step()` (e.g., verifying that shipment arrivals replenish inventory which subsequently allows `process_open()` to fulfill previously backordered customer orders).
3. **System Testing:** Exercises the software end-to-end via the CLI entry point (`python -m scm_sim`) and `build_demo()` scenario execution. Verifies parameter parsing, argument validation, day-by-day logging, and full report generation.
4. **Acceptance Testing:** Evaluates the complete 10-day benchmark scenario against the course problem statement, validating that delays, stockouts, excess inventory, and fill rates are accurately identified and reported.

5.3 Test Types

- **Functional Testing:** Verifies requirement compliance against SCM-F-001 through SCM-F-044.
- **Boundary-Value Analysis:** Tests edge thresholds such as zero quantity, exact capacity matches, boundary reorder levels (level == reorder_point vs. level == reorder_point + 1), and excess boundaries (level > max_level).
- **Regression Testing:** Automated suite executed on every pull request via GitHub Actions to ensure code additions do not degrade existing functionality.
- **Performance Testing (`TC-PERF-01`):** Execution timing of 365-day simulations and 10-day CLI runs ensuring completion within 1.0 second.
- **Reproducibility Testing (`TC-REP-01`):** Verifies that seed-controlled executions yield deterministic, bit-for-bit identical outputs across environments.
- **Security & Process Validation (`TC-SEC-01`, `TC-SEC-02`):** Validates input sanitization, non-negative stock invariants, branch protection rules, and secret absence.

5.4 Entry Criteria

Testing of a component or feature branch may commence only when: 1. Feature module code and corresponding unit tests are authored and syntactically clean. 2. The local Python virtual environment is configured with `pytest` and `pytest-cov`. 3. The baseline continuous integration pipeline on GitHub Actions is active and reporting green on `main`.

5.5 Exit Criteria

The test phase is considered complete and approved for release when: 1. **100% Execution:** All 15 test cases listed in §13.1 have a recorded result; TC-COV-01 remains partial because macOS has not been tested. 2. **Defect-Free Status:** Zero unresolved High or Critical severity defects exist. 3. **Automated Pass Rate:** 100% of automated tests pass (100/100 in the final unit-test report). 4. **Coverage Target:** Code coverage on `src/scm_sim` reaches $\geq 90\%$ (99.79% line, 99.17% branch and 99.66% combined in the final reports). 5. **Quality Review:** Peer review approval is documented for all contributing pull requests.

5.1 Security Validation

As established by the Security Test Engineer (Rishi J Mule), security and data integrity testing validates that the simulator resists invalid inputs, prevents ledger corruption, and enforces development governance standards as specified in SRS §5.1 (SCM-SO-1, SCM-SO-2, SCM-SR-001 through SCM-SR-007).

Security Test Checks and Verification Matrix

The security validation suite verifies eight core security controls across all domain modules:

Security Control ID	Target Invariant / Boundary	Threat Mitigated	Test Verification Strategy	Automated By
SEC-VAL-01: Public Parameter Validation	All public API operations reject non-positive quantities (`qty <= 0`), negative lead times (`lead_time < 0`), invalid reliability metrics (`reliability not in [0.0, 1.0]`), invalid policies, and capacity `<= 0` with `ValueError`.	Data corruption and arithmetic anomalies from malformed inputs.	Parameterized boundary tests asserting `pytest.raises(ValueError)` before state changes.	`tests/test_inventory.pytests/test_orders.pytests/test_shipments.pytests/test_simulator.py`
SEC-VAL-02: Non-Negative Stock Invariant	Warehouse inventory balance can never drop below zero. Attempting to remove more stock than on hand raises `InsufficientStockError` and leaves stock unchanged.	Inventory balance spoofing and phantom deficits.	Negative boundary test asserting `InsufficientStockError` raised when `qty > level(wh, sku)`.	`test_inventory.py::test_remove_too_much_raises`
SEC-VAL-03: Storage Capacity Ceiling Invariant	Total inventory across all SKUs in a registered warehouse can never exceed registered capacity. Breaches raise `CapacityExceededError` and leave stock unchanged.	Storage overflow and unbounded memory consumption.	Boundary tests asserting `CapacityExceededError` when `total + qty > capacity`.	`test_inventory.py::test_add_beyond_capacity_raises_and_leaves_stock_unchanged`

Security Control ID	Target Invariant / Boundary	Threat Mitigated	Test Verification Strategy	Automated By
SEC-VAL-04: Guarded Order Lifecycle	Orders in status `FULFILLED` cannot be cancelled; orders in status `FULFILLED` or `CANCELLED` cannot be re-processed or double-credited.	Double-spending, illegitimate order modifications, and inventory ledger desynchronization.	Negative lifecycle tests attempting cancellation and fulfillment on finalized orders.	`test_orders.py::test_cancel_rulestest_orders.py::test_fulfil_enough_stock`
SEC-VAL-05: Strict Entity Lookup (No Implicit Creation)	Referencing an unregistered supplier, warehouse, or order ID immediately raises `KeyError` without generating phantom objects.	Unvetted entity injection and state hijacking.	Negative lookup tests querying non-existent keys across managers.	`test_shipments.py::test_add_supplier_and_reject_duplicate_and_unknowntest_orders.py::test_cancel_rules`
SEC-VAL-06: State Immutability via Snapshots	Data returned to external callers (such as `Inventory.snapshot()`) must be deep/independent copies.	External state tampering and memory leakage across module boundaries.	Snapshot mutation test asserting that altering the returned dictionary leaves internal `_stock` unchanged.	`test_inventory.py::test_snapshot_returns_independent_copy`
SEC-VAL-07: Branch Protection Enforcement	`main` branch must enforce code reviews, passing CI status checks, linear history, and block direct pushes and force pushes.	Unauthorized, unreviewed, or broken code reaching the production branch.	Manual process audit attempting direct push and unapproved PR merge (TC-SEC-02).	GitHub repository branch ruleset audit
SEC-VAL-08: Dependency & Credential Isolation	Repository contains zero secrets, tokens, or credentials; application makes zero runtime network calls and depends strictly on the standard library.	Supply chain attacks, credential leakage, and runtime remote code execution.	Static code scan for credentials; audit of `requirements.txt` to verify test-only dependencies.	Static analysis and dependency audit (TC-SEC-02)

6. Test Environment

6.1 Hardware Configuration

Test execution is performed across local development workstations and hosted cloud runners: - **Local Developer Machines:** Standard engineering laptops running Windows 11 (64-bit) and Ubuntu 22.04 LTS; minimum 8 GB physical RAM, 2.0 GHz multi-core processor, and 500 MB available disk storage. - **Continuous Integration Runners:** GitHub Actions cloud runners running `ubuntu-latest` with standard 2-core virtual CPU and 7 GB RAM.

6.2 Software Configuration and Runtime

- **Operating Systems:**

    - Local: Microsoft Windows 11 (Build 22631) and Ubuntu Linux 22.04 LTS.
    - Remote CI: Ubuntu Linux (latest release).
- **Python Runtime:** Python 3.11.9 (64-bit). All team members and CI runners operate on Python 3.11 to maintain exact interpreter parity.
- **Virtual Environment:** Dedicated `.venv` created via Python standard library `venv` module to isolate test packages from global system interpreters.

6.3 Test Tools and Utilities

- **Test Framework:** `pytest >= 8.0` (active version: 9.1.1) executing unit, integration, and system test suites.
- **Coverage Tool:** `pytest-cov >= 5.0` (active version: 7.1.0) generating statement and branch coverage metrics.
- **Defect Tracking:** GitHub Issues integrated into the team repository for logging, prioritizing, and resolving defects.
- **Version Control:** Git 2.51.1 coordinating local development and GitHub remote repository synchronization.

6.4 Test Data and Fixtures

All test data is purely synthetic and generated deterministically in memory: - **Pre-configured Simulation Scenario (`build_demo(seed=42)`):** - **Suppliers:** `SUP-RELIABLE` (lead time: 2, reliability: 1.0), `SUP-FLAKY` (lead time: 2, reliability: 0.3). - **Warehouses:** `WH-NORTH` (capacity: 1200), `WH-SOUTH` (capacity: 500). - **SKUs:** `WIDGET`, `GADGET`, `GIZMO`. - **Orders:** 14 scheduled customer orders distributed across simulation days 1 to 10. - **Unit Test Fixtures:** Isolated in-memory objects initialized per test function (`WH1`, `WH2`, `SKU1`, `C-ACME`) with zero persistent filesystem or database state.

7. Test Schedule

Testing runs alongside development. Automated tests were written with each feature and run in CI on pull requests and pushes to `main`. Sections 7.1–7.3 preserve the original baseline plan; §7.4 records the actual schedule and current status. All dates are in 2026 (DD-MM-YYYY).

7.1 Schedule of Test Activities

#	Activity	Start	End	Responsible	Output / milestone	Status
1	Set up test environment: `requirements.txt` (pytest, pytest-cov), `pytest.ini`, CI workflow `.github/workflows/ci.yml` (job `test`)	10-10-2026	10-10-2026	Rohith	**M1: environment and CI ready.** First CI run (#1) green	Done
2	Write unit tests with each feature, run locally and in CI on every PR	10-10-2026	11-10-2026	Each module owner	`test_inventory.py`, `test_orders.py`, `test_shipments.py`, `test_anal`	Done

#	Activity	Start	End	Responsible	Output / milestone	Status
					`ytics.py`	
3	Automated execution on feature PRs: #2 orders, #3 inventory policies, #5 shipments, #6 analytics	10-10-2026	11-10-2026	Each PR author; reviewer checks CI	`test` check green before each merge	Done
4	Integration and system tests for the Simulation Engine and CLI (PR #7)	11-10-2026	11-10-2026	Rohith	`test_simulator.py`; M2 baseline suite completed (historical; see final evidence in `docs/test-reports/`)	Done
5	Test design: write the 15 test-case specifications and this Test Plan	11-10-2026	12-10-2026	All (see §9)	`docs/testplan/` and `docs/testplan/test-cases/`; **M3: Test Plan v1.0 submitted** (docs deadline 12-10-2026)	Done
6	Manual test cases: TC-PERF-01, TC-SEC-02, and manual parts of TC-COV-01 / TC-REP-01	13-10-2026	20-10-2026	Rohanth (PERF), Rohith (SEC-02), Pranav (COV, REP)	Results recorded; Windows and Linux tested, macOS not tested, so SCM-NF-004 remains partial	Done (with macOS portability outstanding)
7	Defect fixing and re-testing for any failures from activity 6	13-10-2026	22-10-2026	Owner of the affected module	No failing test results were reported; no defect-fix cycle was required	Not required
8	Regression testing: full suite re-run by CI on every later PR and every push to `main`	10-10-2026	End of project	Automatic (CI); PR reviewer verifies	Green `test` check on each PR	Ongoing

#	Activity	Start	End	Responsible	Output / milestone	Status
9	Test summary report: metrics from §14, defect summary and exit-criteria check	23-10-2026	09-11-2026	Rohanth (QA Lead) with Rohith	Metrics and test reports recorded; final System Validation Report merged by PR #89	Done

7.2 Milestones

Milestone	Date	Criterion
M1 – Environment and CI ready	10-10-2026	CI workflow merged; first run green
M2 – Initial automated baseline on `main`	11-10-2026	Baseline at `b3702ef`; CI run #20 green (73 passed, 97% coverage). Final suite evidence is in `docs/test-reports/`.
M3 – Test Plan v1.0 submitted	12-10-2026	All Test Plan sections and 15 test-case files merged to `main`
M4 – Test case execution recorded	20-10-2026	All 15 test cases have recorded results; TC-COV-01 is partial because macOS was not tested
M5 – Final system validation report	09-11-2026	Rishi's final report with E1–E6 was merged in PR #89; its requirement matrix is reconciled to SCM-F-034, SCM-F-043 and SCM-F-044

7.3 Dependencies of the Schedule

- Activities 2–4 depend on M1 (CI must exist before PRs can be gated).
- Activity 6 depends on M2 (manual tests run against commit `b3702ef` or a later green `main`).
- Activity 9 depends on M4 and on all defects from activity 7 being closed or accepted.
- If testing is suspended (§12), the end dates of activities 6–9 move back by the length of the suspension; M3 is fixed by the course deadline.

7.4 Actual Schedule

Phase / Activity	Start	End	Outputs & Milestones	Status
Sprint 1	01-11-2026	06-11-2026	Packaging, order ID validation (#68), back-orders first (#69), arrivals overflow fix (#70), per-item thresholds (#71), Sprint 1 video (#75)	Done
Testing week	02-11-2026	06-11-2026	v1 test reports; manual test cases TC-REP-01, TC-PERF-01, and TC-SEC-02 executed and passed on 05-11-2026	Done

Phase / Activity	Start	End	Outputs & Milestones	Status
Sprint 2	07-11-2026	09-11-2026	Run summary (#79), CSV export (#80), coverage gap tests (#81, #84), test case updates (#83), CI report workflows (#82), unit and coverage reports	Done
Release v1.0	09-11-2026	Pending	Deviations and final test/coverage/system-validation evidence are available; GitHub release/freeze issue #58 and Sprint 2 demo video issue #57 remain open	In progress

8. Test Deliverables

The verification process produces a structured set of technical deliverables to ensure transparent validation, traceability, and reproducibility:

Deliverable Artifact	Format / Location	Description & Contents
Test Plan Document	Markdown (`docs/testplan/`) & DOCX (`docs/final/`)	Comprehensive verification strategy defining scope, approach, environment, schedules, roles, and test cases conforming to IEEE 829.
Test Case Specifications	Markdown (`docs/testplan/test-cases/*.md`)	Granular test case specifications (TC-INV-01 through TC-REP-01) detailing objectives, preconditions, test data, steps, expected results, and pass/fail criteria.
Automated Test Suite	Python files (`tests/test_*.py`)	Complete executable pytest test suite containing 100 automated unit, integration, and system tests with zero external test fixtures.
CI Execution Logs	GitHub Actions Console Logs	Automated job logs for each run of the `test` workflow on Ubuntu runners, capturing timestamped execution outputs and exit statuses.
Code Coverage Reports	Terminal output & HTML summaries (`pytest-cov`)	Statement and branch coverage reports generated automatically during test execution, demonstrating compliance with the $\geq 90\%$ requirement (final evidence: 99.79% line and 99.17% branch coverage).
Defect Reports & Tracking	GitHub Issues	Documented records of any functional bugs, edge-case failures, or coverage gaps identified during

Deliverable Artifact	Format / Location	Description & Contents
		review, tracked through resolution.
Test Summary Report	Section 14 of Test Plan & Release Notes	High-level executive summary presenting final test metrics, total passed tests, CI stability rate, and release readiness sign-off.

9. Roles and Responsibilities

9.1 Team Roles

Name	SRN	GitHub	Test Plan role	Code module owned	Responsibilities in testing
Rohith Karning	<SRN>	rohithkarning-debug	**Product Owner** (also Test Engineer)	Inventory; Simulation Engine, CLI and CI Pipeline (integration); repository owner	Owns the requirements baseline and the requirements registry; decides whether a requirement is accepted. Set up the test environment and CI. Writes and executes TC-INV-01, TC-INV-02, TC-SIM-01 and TC-SEC-02. Approves the Test Plan and the final test summary.
Rohanth Ganesha C	<SRN>	rohanth3	**QA Lead** (also Test Engineer)	Order Manager	Owns this Test Plan's approach, entry and exit criteria and suspension criteria. Tracks test progress and defects, and writes the test summary report (§7, M5). Writes and executes TC-ORD-01, TC-ORD-02 and TC-PERF-01.
Rishi J Mule	<SRN>	Rishi-Jayprakash06	**Security Test Engineer** (also Test Engineer)	Shipment Manager	Owns security validation (§5.1) and the security requirements SCM-SR-001 to SCM-SR-007. Writes and executes TC-SHP-01, TC-SHP-

Name	SRN	GitHub	Test Plan role	Code module owned	Responsibilities in testing
					02, TC-SEC-01 and the CLI system test TC-SIM-02.
Pranav Achar	<SRN>	achar-pranav	**Dev Lead** (also Test Engineer)	Analytics	Owns code quality and the coverage target (SCM-NF-003). Checks that tests stay in step with the code, and fixes defects in Analytics. Writes and executes TC-ANA-01, TC-ANA-02, TC-REP-01 and TC-COV-01.

Every team member is a **Test Engineer** for the test cases they own: they write the test-case specification, keep the automated pytest tests for it passing, execute any manual steps, and record the actual result and status.

9.2 Test Case Ownership

Owner	Test cases	Requirements covered
Rohith Karning	TC-INV-01, TC-INV-02, TC-SIM-01, TC-SEC-02	SCM-F-001 – F-004, SCM-F-041, SCM-F-042, SCM-SR-005, SCM-SR-006
Rohanth Ganesha C	TC-ORD-01, TC-ORD-02, TC-PERF-01	SCM-F-010 – F-013, SCM-NF-001
Rishi J Mule	TC-SHP-01, TC-SHP-02, TC-SEC-01, TC-SIM-02	SCM-F-020 – F-023, SCM-SR-001 – SR-004, SCM-SR-007, SCM-F-040, SCM-F-043, SCM-NF-005
Pranav Achar	TC-ANA-01, TC-ANA-02, TC-REP-01, TC-COV-01	SCM-F-030 – F-033, SCM-NF-002, SCM-NF-003, SCM-NF-004

9.3 Responsibility Matrix (RACI)

R = Responsible (does the work), A = Accountable (approves), C = Consulted, I = Informed.

Activity	Rohith (Product Owner)	Rohanth (QA Lead)	Rishi (Security Test Eng.)	Pranav (Dev Lead)
Requirements baseline and registry	A, R	C	C	C
Test Plan approach, entry and exit criteria	C	A, R	C	C
Test environment and CI pipeline	A, R	I	C	C
Writing test cases for own module	R	R	R	R

Activity	Rohith (Product Owner)	Rohanth (QA Lead)	Rishi (Security Test Eng.)	Pranav (Dev Lead)
Security validation (§5.1)	C	I	A, R	C
Performance test (TC-PERF-01)	I	A, R	I	C
Coverage and reproducibility (TC-COV-01, TC-REP-01)	I	C	I	A, R
Branch-protection and secrets check (TC-SEC-02)	R	I	A	I
Code review of PRs (every PR needs one approval from a non-author)	R	R	R	R
Defect fixing	R (own modules)	R (own module)	R (own module)	A, R (own module)
Test summary report	C	A, R	C	C
Final approval of the Test Plan (§15)	A	R	I	R

9.4 Review Responsibilities

Every test-plan and test-case file reaches `main` through a pull request that needs one approval from a teammate other than its author, plus a green `test` CI check. For the Test Plan, reviewers rotate as follows:

PR author	Test Plan PR reviewed by
Rohith	Pranav
Rohanth	Rohith
Rishi	Rohanth
Pranav	Rishi

The reviewer checks that every requirement and test-case ID matches `docs/requirements-registry.md`, that each automated test case names pytest tests that actually exist in `tests/`, and leaves at least one review comment before approving.

10. Risks and Mitigation

The following risk matrix identifies potential quality, execution, and integration risks identified during test planning, along with proactive mitigation strategies and contingency actions:

Risk ID	Risk Description	Severity	Likelihood	Mitigation Strategy	Contingency Plan
RSK-01	**Nondeterministic Test Failures (Flaky Tests):** Stochastic delay generation causes intermittent test failures when using	High	High	Enforce dependency injection of seeded pseudorandom number generators (`random.Random(42)`) in	If a test fails unpredictably, inspect whether global `random` was invoked instead of the injected `self.rng` instance.

Risk ID	Risk Description	Severity	Likelihood	Mitigation Strategy	Contingency Plan
	unseeded random numbers.			`ShipmentManager` and `Simulation`. In unit tests, either pass explicit `delay` parameters or explicit seed integers.	
RSK-02	**Continuous Integration Service Outage:** GitHub Actions service downtime prevents automated check execution and blocks PR merges.	Medium	Low	Maintain full local test parity. All team members configure `.venv` locally with identical dependencies (`pytest`, `pytest-cov`) and must achieve 100% pass rates locally before opening PRs.	Team lead temporarily verifies test execution via signed local run logs if emergency integration is mandated.
RSK-03	**Module Interface Incompatibility:** Parallel development by four engineers leads to signature mismatches during integration into `simulator.py`.	High	Medium	Freeze and publish exact public API signatures, parameters, exceptions, and return types in `SCM_PROJECT_CONTEXT.md` §4.5 prior to module construction.	Dedicated integration test suite (`tests/test_simulator.py`) exercises end-to-end multi-module flows on every PR.
RSK-04	**Cross-Platform Inconsistencies:** Discrepancies between Windows local developer environments and Linux CI runners (e.g., line endings, CLI shell parsing).	Medium	Low	Rely exclusively on cross-platform Python standard library modules (`argparse`, `sys`). Run full automated regression on `ubuntu-latest` CI for every Windows push.	Author cross-platform tests (`TC-COV-01`) verifying CLI invocation syntax across both Windows PowerShell and Linux bash.
RSK-05	**Test State Pollution:** Tests modifying	High	Low	Pure object-oriented encapsulation:	Pytest executes tests with randomized

Risk ID	Risk Description	Severity	Likelihood	Mitigation Strategy	Contingency Plan
	shared global state, causing cascading order-dependent test failures.			every test instantiates fresh, isolated object instances (`Inventory()`, `OrderManager(inv)`). Zero module-level mutable variables.	order during test auditing to verify complete independence.

11. Assumptions and Dependencies

11.1 Assumptions

The test effort defined in this plan rests on the assumptions below. Each is stated so that a reviewer can judge whether the plan is still valid if the assumption fails, and so that a failure of an assumption is not mistaken for a defect in the system under test.

#	Assumption	Basis	Consequence if it does not hold
A1	**The source code under test is the version merged into `main` at commit `b3702ef`.** All test items, interfaces and baselines in §2 refer to this commit.	The code is built and merged; `main` is at `ef435c7` with this baseline among its history.	Any change after the baseline invalidates the affected interface listing in §2.4 and requires the affected test cases to be re-executed before the results are reported.
A2	**One supplier serves each item, and demand is known in advance.** A scenario links one supplier to each per-item policy, and orders are scheduled for explicit future days.	SRS §2.6 states these as design assumptions of the simulated network.	If multi-sourcing or demand uncertainty were introduced, SCM-F-042 and SCM-F-012 would need to specify a selection rule that does not currently exist, and TC-SIM-01 would no longer be sufficient for them.
A3	**Delays only ever make a shipment later.** The delay model adds 1–3 days to the arrival day and never brings it forward.	SCM-F-022 defines the model as an addition only; `rng.random() > reliability` yields 0 or a positive delay.	Early-arrival cases would need their own analytics behaviour. `find_delayed_shipments` would still exclude them (`days_late > 0`), so TC-ANA-01 would remain valid, but the days-late semantics documented in SAD §4.3.4 would need revision.
A4	**Randomness is drawn from a single injected**	SAD §3.6; the PRNG is created once in	SCM-NF-002 becomes unverifiable, because no

#	Assumption	Basis	Consequence if it does not hold
	`random.Random(seed)`. No module calls the module-level `random` functions.	`Simulation` and injected into `ShipmentManager`.	run would be reproducible. This assumption is directly tested by `test_same_seed_gives_identical_reports` in TC-REP-01, so a violation would be detected rather than assumed away.
A5	**The Python runtime is version 3.11 or later and provides the standard-library modules the system uses.** The system imports only `dataclasses`, `enum`, `random` and `argparse` at runtime.	SRS §2.4, §5 (SCM-NF-004); `requirements.txt`.	Test results from one interpreter version do not transfer to another. This is why the portability half of TC-COV-01 is executed on a second host rather than inferred from CI.
A6	**The CI environment is representative enough to execute the automated suite.** GitHub Actions `ubuntu-latest` with Python 3.11 runs `pytest -v --cov=scm_sim` and its result is treated as the platform-neutral result.	SCM-NF-003, SCM-NF-004; the workflow in `.github/workflows/ci.yml`.	A failure specific to a platform CI does not cover, such as Windows path or line-ending behaviour, would go undetected. This is the gap the manual portability run in TC-COV-01 exists to close.
A7	**All test data is synthetic.** Every identifier used in a test case — warehouses `WH-NORTH`, `WH-SOUTH`, SKUs `WIDGET`, `GADGET`, `GIZMO`, customers `C-ACME`, suppliers `SUP-RELIABLE`, `SUP-FLAKY`, orders `ORD-0001`, shipments `SHP-0001` — is invented for the purpose.	No real supply chain, customer or supplier data exists in the project, and none is required.	If real data were introduced, SCM-SR-006 would need review, because the repository is assumed to hold no confidential material. The assumption is verified by the secrets scan in TC-SEC-02.
A8	**GitHub branch protection is configured as specified.** The rules for `main` are those enumerated in SCM-SR-005.	The team's documented workflow; the rules are not visible in the repository itself.	TC-SEC-02 must inspect Settings → Branches rather than trust this assumption, which is why it is defined as a manual check rather than an automated one.
A9	**A performance measurement on a standard development laptop is representative.**	SCM-NF-001 specifies a "standard laptop with Python 3.11" and records baselines of roughly 0.02	If a slower or faster host is used, the measured time must be reported together with the host

#	Assumption	Basis	Consequence if it does not hold
	The reference platform is an ordinary x86-64 laptop rather than a constrained or server-grade machine.	s and 0.10 s.	description, and the 1 s budget evaluated against that context.
A10	**The requirements registry is stable and correct.** The IDs in `docs/requirements-registry.md` match SRS, SAD and this plan, and no requirement ID is changed or added without notifying the repository owner.	The registry is the single source of truth agreed by the team.	Every traceability table in all three documents would need re-derivation. Changes to IDs are therefore treated as a document-level event rather than an editing convenience.

11.2 Dependencies

The test effort cannot be completed independently of the following. Each dependency states what is required, who provides it and what happens if it is unavailable.

#	Dependency	Provided by	Needed for	Effect if unavailable
D1	**A working Python 3.11 environment** with `pytest` and `pytest-cov` installed from `requirements.txt`.	Each team member; CI provisions it automatically.	Execution of every automated test case.	Automated execution stops. Results already recorded for commit `b3702ef` remain valid for that commit only.
D2	**The `test` CI job on GitHub Actions** (`ubuntu-latest`, Python 3.11, `pytest -v --cov=scm_sim`).	Repository owner (Rohith Karning).	Continuous regression testing of every pull request; the SCM-NF-003 coverage gate; the SCM-SR-005 merge condition.	No merge is permitted, because the required check cannot be satisfied. Testing becomes manual-only, which is not an acceptable fallback for this project.
D3	**GitHub branch protection on `main`.**	Repository owner (Rohith Karning), configured in repository settings.	SCM-SR-005; TC-SEC-02.	The merge gate is unenforced. SCM-SR-005 would fail and, with it, the integrity objective SCM-SO-2.
D4	**A second, non-CI host running Windows** for the portability execution.	A team member with a Windows 11 laptop (see Test Plan §6).	The manual half of TC-COV-01 covering SCM-NF-004.	SCM-NF-004 stays at status N. Portability would be evidenced for Linux only, which is insufficient to claim support for Windows and macOS.

#	Dependency	Provided by	Needed for	Effect if unavailable
D5	**Wall-clock timing capability** (`time` on bash, `Measure-Command` in PowerShell) and three consecutive runs per measurement.	Standard on both host operating systems; no purchase required.	TC-PERF-01 covering SCM-NF-001.	Performance cannot be evidenced. A single run would not distinguish a genuine pass from timing noise, so the three-run requirement is retained.
D6	**The test suite itself.** The original 73-test baseline was expanded to 100 automated tests across the unit, integration, CLI and packaging suites.	All four team members, one owner per module.	Every automated test case in §13.	A test case whose automation is missing or failing cannot be reported as passed, regardless of whether manual reasoning suggests the behaviour is correct.
D7	**The requirements registry and the three specifications** (SRS v1.0, SAD v1.0, this Test Plan v1.0).	Product Owner (Rohith Karning) as owner of the registry.	Traceability in §13, the feature list in §3 and the status column of SRS §8.	Traceability cannot be asserted, and the exit criteria in §5.4 — which require every planned test case to have been executed against a named requirement — cannot be evaluated.
D8	**Access to repository settings** for the branch-protection and secrets inspection.	Repository owner.	The manual steps of TC-SEC-02.	SCM-SR-005 and SCM-SR-006 cannot be evidenced by inspection and remain unverified.
D9	**Documentation of the single demo command** (`PYTHONPATH=src python -m scm_sim --days 10 --seed 42`).	README of the repository.	The usability half of SCM-NF-005, exercised in TC-SIM-02.	A new user cannot run the demo unaided, so SCM-NF-005 fails even though the CLI itself behaves correctly.
D10	**Team availability for peer review and for the remaining manual executions**, scheduled to complete by 20-10-2026 (Test Plan §7).	All four team members.	Reviewer rotation, and the six requirements still at status N in SRS §8.	Those requirements remain unverified and the project cannot report full acceptance, even though no code defect is implied.

11.3 Assumptions and Dependencies Not Applicable

Two categories that would normally appear in this section do not apply to this project, and are recorded here so that their absence is deliberate:

- **No test data management dependency.** There is no database, no production dataset and no data-masking process. All test data is constructed in code by `build_demo()` or by small in-test fixtures, and is entirely synthetic.
- **No external service or network dependency.** The simulator makes no network calls and depends on no external service, API or database at test time, so there is nothing to stub, mock or rate-limit, and no third-party outage can affect test execution.

12. Suspension and Resumption Criteria

This section outlines the operational conditions governing when testing activities must be paused and when they may safely resume.

12.1 Suspension Criteria

Testing activities and pull request evaluations shall be formally suspended under any of the following conditions: 1. **Broken Continuous Integration Pipeline:** The `test` job on GitHub Actions fails on the `main` branch, indicating an unhandled regression or broken baseline environment. 2. **High Test Blockage Threshold:** More than 30% of automated test cases are failing or blocked due to breaking interface changes or uncoordinated domain signature modifications. 3. **Core Dependency Invalidation:** Critical foundational modules (such as `Inventory`) fail core state invariant checks (e.g., negative stock levels allowed or capacity exceptions unhandled), compromising downstream module verification. 4. **Environment Unavailability:** Severe disruptions to the primary developer environments or GitHub Actions CI runner infrastructure.

12.2 Resumption Criteria

Testing activities shall resume once all blocking criteria have been addressed: 1. **CI Pipeline Restored:** A fix or revert is merged to `main`, restoring a green check status across the full automated test suite. 2. **Defect Rectification:** Blocking interface discrepancies are resolved, verified by local execution of `pytest -v`, and validated to pass with zero regressions. 3. **Baseline Re-verification:** The test suite pass rate returns to 100%, and coverage metrics are confirmed $\geq 90\%$ before downstream feature evaluation continues.

13. Test Case Management and Traceability

13.1 Test Case Register

Fifteen test cases are documented. Automated cases run in CI; mixed and manual cases include their execution evidence in the individual test-case records. The register below is the index; the individual test cases are held in `docs/testplan/test-cases/`.

TC ID	Title	Covers	Type	Owner	Automation status
TC-INV-01	Stock tracking, mutation validation and warehouse capacity	SCM-F-001, SCM-F-002, SCM-SR-001, SCM-SR-002	Functional	Rohith Karning	Automated (see §13.3)
TC-INV-02	Per-item stock policy, reorder suggestions and snapshots	SCM-F-003, SCM-F-004, SCM-SR-001, SCM-SR-007	Functional	Rohith Karning	Automated (see §13.3)
TC-ORD-01	Order creation, sequential identifiers and all-or-nothing fulfilment	SCM-F-010, SCM-F-011	Functional	Rohanth Ganesha C	Automated (see §13.3)
TC-ORD-02	Back-order retry ordering,	SCM-F-012, SCM-F-013	Functional	Rohanth Ganesha C	Automated (see §13.3)

TC ID	Title	Covers	Type	Owner	Automation status
	cancellation rules and status filtering				
TC-SHP-01	Supplier validation and shipment dispatch	SCM-F-020, SCM-F-021	Functional	Rishi J Mule	Automated (see §13.3)
TC-SHP-02	Delay modelling, arrival processing and delay detection	SCM-F-022, SCM-F-023	Functional	Rishi J Mule	Automated (see §13.3)
TC-SIM-01	Daily time-step ordering and automatic replenishment	SCM-F-023, SCM-F-041, SCM-F-042	Integration	Rohith Karning	Automated (see §13.3)
TC-SIM-02	Scenario configuration, demo detection and CLI behaviour	SCM-F-034, SCM-F-040, SCM-F-043, SCM-F-044, SCM-NF-005	System	Rishi J Mule	Automated (see §13.3)
TC-ANA-01	Delayed-shipment and inventory-exception detection	SCM-F-030, SCM-F-031	Functional	Pranav Achar	Automated (see §13.3)
TC-ANA-02	Order metrics, report composition, formatting and run summary	SCM-F-032, SCM-F-033, SCM-F-034	Functional	Pranav Achar	Automated (see §13.3)
TC-REP-01	Seeded reproducibility of reports and output	SCM-NF-002	Reliability	Pranav Achar	Mixed: 1 automated pytest test plus a manual CLI comparison
TC-COV-01	Coverage gate and cross-platform portability	SCM-NF-003, SCM-NF-004	Maintainability	Pranav Achar	Mixed: CI job result plus a manual run on Windows and Linux
TC-PERF-01	Runtime of the 365-day run and the 10-day CLI demo	SCM-NF-001	Performance	Rohanth Ganesha C	Manual
TC-SEC-01	Input validation, state invariants and reference integrity	SCM-SR-001, SCM-SR-002, SCM-SR-003, SCM-SR-004, SCM-SR-007	Security	Rishi J Mule	Automated (9 pytest tests across 4 modules)
TC-SEC-02	Branch	SCM-SR-005,	Security	Rohith Karning	Manual

TC ID	Title	Covers	Type	Owner	Automation status
	protection, secrets absence and dependency hygiene	SCM-SR-006	(process)		

13.2 Requirements Traceability Matrix (Requirement to Test Case)

The matrix below traces every requirement in the registry to the test case or cases that verify it. It is the test-side counterpart of the RTM in SRS §8.

Status codes: A = accepted with passing evidence; P = partially validated because some required evidence is outstanding; N = not verified because required evidence is absent.

Req ID	Requirement short	SRS section	Test case(s)	Test level	Result	Status
SCM-F-001	Stock per (warehouse, SKU) with validated add/remove	§4.1	TC-INV-01	Unit	Pass (CI run #20, `b3702ef`)	A
SCM-F-002	Warehouse capacity with `CapacityExceededError`	§4.1	TC-INV-01	Unit	Pass (CI run #20, `b3702ef`)	A
SCM-F-003	Per-item policy, `needs_reorder`, `is_excess`	§4.1	TC-INV-02	Unit	Pass (CI run #20, `b3702ef`)	A
SCM-F-004	Reorder suggestions and independent snapshot	§4.1	TC-INV-02	Unit	Pass (CI run #20, `b3702ef`)	A
SCM-F-010	Order creation with sequential IDs	§4.2	TC-ORD-01	Unit	Pass (CI run #20, `b3702ef`)	A
SCM-F-011	Fulfil in full or back-order, no partial fulfilment	§4.2	TC-ORD-01	Unit	Pass (CI run #20, `b3702ef`)	A
SCM-F-012	Retry open orders oldest-first	§4.2	TC-ORD-02	Unit	Pass (CI run #20, `b3702ef`)	A
SCM-F-013	Cancel, look up by ID, list by status	§4.2	TC-ORD-02	Unit	Pass (CI run #20, `b3702ef`)	A
SCM-F-020	Supplier registration with lead	§4.3	TC-SHP-01	Unit	Pass (CI run #20, `b3702ef`)	A

Req ID	Requirement short	SRS section	Test case(s)	Test level	Result	Status
	time and reliability					
SCM-F-021	Dispatch with sequential IDs and expected day	§4.3	TC-SHP-01	Unit	Pass (CI run #20, `b3702ef`)	A
SCM-F-022	Stochastic delay model from reliability	§4.3	TC-SHP-02	Unit	Pass (CI run #20, `b3702ef`)	A
SCM-F-023	Arrival processing and late-shipment listing	§4.3	TC-SHP-02, TC-SIM-01	Unit, Integration	Pass (CI run #20, `b3702ef`)	A
SCM-F-030	Delayed shipments with days late, worst first	§4.4	TC-ANA-01	Unit	Pass (CI run #20, `b3702ef`)	A
SCM-F-031	Stockouts, low stock and excess with surplus	§4.4	TC-ANA-01	Unit	Pass (CI run #20, `b3702ef`)	A
SCM-F-032	Order metrics and fill rate to 2 dp	§4.4	TC-ANA-02	Unit	Pass (final unit-test report: 100 tests, commit `30702f8`)	A
SCM-F-033	Daily report composition and formatting	§4.4	TC-ANA-02	Unit	Pass (final unit-test report: 100 tests, commit `30702f8`)	A
SCM-F-034	Multi-day run summary and `--summary` display	§4.4	TC-ANA-02, TC-SIM-02	Unit, System	Pass (final unit-test report: 100 tests, commit `30702f8`; E5 is recorded in the System Validation Report)	A
SCM-F-040	Scenario configuration with validation	§4.5	TC-SIM-02	System	Pass (final unit-test report: 100 tests, commit `30702f8`)	A
SCM-F-041	Fixed-order daily step and report	§4.5	TC-SIM-01	Integration	Pass (final unit-test report: 100	A

Req ID	Requirement short	SRS section	Test case(s)	Test level	Result	Status
	retention				tests, commit `30702f8`)	
SCM-F-042	Automatic trimmed replenishment	§4.5	TC-SIM-01	Integration	Pass (final unit-test report: 100 tests, commit `30702f8`)	A
SCM-F-043	CLI with exit code 0 and usage error at 2	§4.5	TC-SIM-02	System	Pass (final unit-test report: 100 tests, commit `30702f8`)	A
SCM-F-044	Daily metrics CSV export with `--csv` PATH	§4.5	TC-SIM-02	System	Pass (final unit-test report: 100 tests, commit `30702f8`; E6 is recorded in the System Validation Report)	A
SCM-NF-001	365-day run and 10-day CLI demo each ≤ 1.0 s	§5	TC-PERF-01	Performance	Pass; three measured trials for each run on Windows 11 / Python 3.11.9	A
SCM-NF-002	Same seed and scenario give identical output	§5	TC-REP-01	Reliability	Pass; automated report equality and manual CLI comparisons recorded	A
SCM-NF-003	$\geq 90\%$ line coverage, all tests pass before merge	§5	TC-COV-01	Maintainability	Pass; 100 tests, 99.79% line, 99.17% branch, 99.66% combined coverage	A
SCM-NF-004	Runs on Windows, Linux, macOS; standard library only	§5	TC-COV-01	Portability	Windows and Linux tested; macOS not tested	P
SCM-NF-005	One documented command; counts and	§5	TC-SIM-02	System	Pass; package command, summary/CSV options and	A

Req ID	Requirement short	SRS section	Test case(s)	Test level	Result	Status
	"No issues"; `--help`				CLI behavior verified	
SCM-SR-001	Input validation before any state change	§5.1.2	TC-INV-01, TC-INV-02, TC-SEC-01	Security	Pass (CI run #20, `b3702ef`)	A
SCM-SR-002	Stock invariants and unchanged state on violation	§5.1.2	TC-INV-01, TC-SEC-01	Security	Pass (CI run #20, `b3702ef`)	A
SCM-SR-003	Guarded order status transitions	§5.1.2	TC-SEC-01	Security	Pass (CI run #20, `b3702ef`)	A
SCM-SR-004	Unknown references rejected with `KeyError`	§5.1.2	TC-SEC-01	Security	Pass (CI run #20, `b3702ef`)	A
SCM-SR-005	Protected `main` branch rules	§5.1.2	TC-SEC-02	Security (process)	Pass; branch rules, push rejection and blocked merge recorded in `docs/test plan/results/TC-SEC-02.md`	A
SCM-SR-006	No secrets, no network calls, no runtime dependencies	§5.1.2	TC-SEC-02	Security (process)	Pass; repository scans and dependency inspection recorded in `docs/test plan/results/TC-SEC-02.md`	A
SCM-SR-007	Returned data is an independent copy	§5.1.2	TC-INV-02, TC-SEC-01	Security	Pass (CI run #20, `b3702ef`)	A

The two security objectives require no test case of their own: SCM-SO-1 is evidenced by the accepted results for SCM-SR-001 to SCM-SR-004 and SCM-SR-007, and SCM-SO-2 is evidenced by the accepted results for SCM-SR-005 and SCM-SR-006.

The `CI run #20` entries above preserve the original baseline evidence (73 automated tests). The final suite result is 100 / 100 passed, as recorded in the final unit-test report; final line and branch coverage evidence is recorded in the coverage reports and branch-coverage report.

13.3 Reverse Traceability (Test Case to Requirement and Automation)

TC ID	Requirements	Automated by	Manual part
TC-INV-01	SCM-F-001, SCM-F-002, SCM-SR-001, SCM-SR-002	`tests/test_inventory.py`: `test_add_and_level`, `test_remove_reduces_stock`, `test_remove_too_much_raises`, `test_remove_rejects_non_positive_quantity`, `test_add_up_to_exact_capacity_is_allowed`, `test_add_beyond_capacity_raises_and_leaves_stock_unchanged`, `test_unregistered_warehouse_has_unlimited_capacity`; supporting: `test_stockout_detected`, `test_register_warehouse_rejects_non_positive_capacity`, `test_total_in_warehouse_sums_all_skus`, `test_existing_behaviour_unchanged_with_capacity_registered`	–
TC-INV-02	SCM-F-003, SCM-F-004, SCM-SR-001, SCM-SR-007	`tests/test_inventory.py`: `test_stock_policy_rejects_invalid_values`, `test_needs_reorder_at_boundary`, `test_is_excess_only_above_max_level`, `test_reorder_suggestions_sorted_and_only_tracked_items`, `test_snapshot_returns_independent_copy`; supporting: `test_stock_policy_defaults`, `test_stock_policy_allows_max_level_equal_to_reorder_po	–

TC ID	Requirements	Automated by	Manual part
		int, test_get_policy_returns_default_when_unset, test_set_policy_is_per_warehouse_and_sku, test_reorder_suggestions_empty_for_new_inventory	
TC-ORD-01	SCM-F-010, SCM-F-011	tests/test_orders.py: test_create_order_id_sequence, test_invalid_quantity, test_fulfil_enough_stock, test_backorder_when_short, test_rejected_order_does_not_consume_an_id, test_order_dataclass_rejects_non_positive_quantity_directly	–
TC-ORD-02	SCM-F-012, SCM-F-013	tests/test_orders.py: test_backorder_later_fulfilled_by_process_open, test_oldest_first_priority, test_cancel_rules, test_get_unknown_order_raises_key_error, test_fulfilled_or_cancelled_orders_are_never_reprocessed; tests/test_simulator.py: test_older_backorder_is_served_before_new_order_on_same_day	–
TC-SHP-01	SCM-F-020, SCM-F-021	tests/test_shipments.py: test_supplier_validation, test_add_supplier_and_reject_duplica	–

TC ID	Requirements	Automated by	Manual part
		te_and_unknown, test_expected_day_calculated_from_lead_time, test_shipment_id_sequence_and_in_transit_filtering, test_reject_non_positive_quantity	
TC-SHP-02	SCM-F-022, SCM-F-023	tests/test_shipments.py: test_on_time_arrival_adds_stock, test_delayed_shipment_not_delivered_early, test_delayed_detection_both_in_transit_late_and_delivered_late, test_reliability_one_means_never_delayed, test_probabilistic_delay_with_zero_reliability, test_capacity_breach_does_not_abort_other_arrivals; tests/test_simulator.py: test_blocked_arrival_is_logged_and_delivered_once_space_frees	–
TC-SIM-01	SCM-F-023, SCM-F-041, SCM-F-042	tests/test_simulator.py: test_step_advances_day_counter, test_shipment_arrives_after_lead_time_and_increases_stock, test_large_order_is_backordered_then_fulfilled_after_replenishment, test_low_stock_triggers_exactly_one_reorder_while_in_transit, test_reorder_is_trimmed_to_fit_warehouse_capacity,	–

TC ID	Requirements	Automated by	Manual part
		test_older_backorder_is_served_before_new_order_on_same_day	
TC-SIM-02	SCM-F-034, SCM-F-040, SCM-F-043, SCM-F-044, SCM-NF-005	tests/test_simulator.py: test_schedule_order_validates_input, test_demo_detects_delay_stockout_backorder_and_excess, test_cli_main_runs, test_cli_main_summary_flag, test_metrics_rows_has_one_flat_row_per_day, test_cli_csv_export, test_cli_csv_and_summary_flags, test_cli_rejects_zero_days, test_cli_runs_as_module; tests/test_packaging.py: test_installed_scm_sim_command_runs, test_pyproject_declares_scm_sim_console_script, test_package_has_no_runtime_dependencies	–
TC-ANA-01	SCM-F-030, SCM-F-031	tests/test_analytics.py: test_in_transit_late_uses_current_day, test_delivered_late_uses_actual_day_not_current_day, test_on_time_and_early_are_excluded, test_in_transit_not_yet_due_is_excluded, test_arrival_on_day_zero_is_not_treated_as_in_transit, test_sorted_by_days_late_descending, test_stockouts_onl	–

TC ID	Requirements	Automated by	Manual part
		y_exact_zero_and_sorted, test_low_stock_boundaries_zero_excluded_and_20_included, test_low_stock_respects_custom_reorder_point, test_excess_boundaries_500_excluded_and_501_included, test_excess_surplus_with_custom_max_level, test_empty_stock_gives_empty_results, test_low_stock_and_excess_use_each_items_own_policy, test_items_without_policy_fall_back_to_global_thresholds, test_build_report_passes_policies_through; tests/test_simulator.py: test_report_uses_per_item_policy_thresholds	
TC-ANA-02	SCM-F-032, SCM-F-033, SCM-F-034	tests/test_analytics.py: test_order_metrics_counts_with_string_statuses, test_order_metrics_accepts_enum_statuses, test_enum_and_string_statuses_can_be_mixed, test_fill_rate_excludes_cancelled_orders, test_fill_rate_is_zero_with_no_orders, test_fill_rate_is_zero_when_all_cancelled, test_build_report_has_all_keys,	–

TC ID	Requirements	Automated by	Manual part
		`test_build_report_contents`, `test_build_report_passes_custom_thresholds_through`, `test_format_report_has_section_headers_and_counts`, `test_format_report_prints_no_issues_for_empty_sections`, `test_summarize_run_and_format`, `test_summarize_run_empty_history`, `test_order_metrics_unknown_status`	
TC-REP-01	SCM-NF-002	`tests/test_simulator.py::test_same_seed_gives_identical_reports`	Run the CLI twice with `--seed 42` and compare the output; then run with `--seed 7` and confirm the delays differ
TC-COV-01	SCM-NF-003, SCM-NF-004	CI `test` job (`pytest -v --cov=scm_sim --cov-branch --cov-fail-under=90`): 100 tests passed; 99.79% line, 99.17% branch, 99.66% combined coverage	Windows 11 and Linux tested; macOS not tested
TC-PERF-01	SCM-NF-001	–	Time `build_demo().run(365)` and the 10-day CLI demo with `time` / `Measure-Command`, 3 runs each
TC-SEC-01	SCM-SR-001, SCM-SR-002, SCM-SR-003, SCM-SR-004, SCM-SR-007	`tests/test_inventory.py`: `test_remove_rejects_non_positive_quantity`, `test_remove_too_much_raises`, `test_register_warehouse_rejects_non_positive_capacity`, `test_add_beyond_capacity_raises_and_leaves_stock_unchanged`, `test_stock_policy_rejects_invalid_values`, `test_snapshot_retu	–

TC ID	Requirements	Automated by	Manual part
		`rns_independent_copy`; `tests/test_orders.py`: `test_invalid_quantity`, `test_rejected_order_does_not_consume_an_id`, `test_order_dataclass_rejects_non_positive_quantity_directly`, `test_get_unknown_order_raises_key_error`, `test_cancel_rules`, `test_fulfilled_or_cancelled_orders_are_never_reprocessed`; `tests/test_shipments.py`: `test_supplier_validation`, `test_add_supplier_and_reject_duplicate_and_unknown`, `test_reject_non_positive_quantity`; `tests/test_simulator.py`: `test_schedule_order_validates_input`, `test_set_policy_rejects_unknown_supplier`	
TC-SEC-02	SCM-SR-005, SCM-SR-006	–	Attempt a direct push to `main`; attempt to merge a PR without approval; inspect Settings → Branches; search the repository for tokens and secrets; check `requirements.txt` for test-only dependencies

13.4 Coverage Analysis

Metric	Value
Requirements in the registry	34 (22 functional, 5 non-functional, 7 security)
Requirements traced to at least one test case	34
Functional requirements	22 total; 22 accepted
Non-functional requirements	5 total; 4 accepted, 1 partially validated

Metric	Value
Security requirements	7 total; 7 accepted
Total requirement status	34 total; 33 accepted, 1 partially validated, 0 not verified
Requirements untraced	0
Test cases planned	15
Test cases fully automated	11
Test cases mixed (automated plus manual)	2 (TC-REP-01, TC-COV-01)
Test cases fully manual	2 (TC-PERF-01, TC-SEC-02)
Test items under test	7 (Test Plan §2)
Test items with at least one test case	7
Requirements per test case (average)	2.7 (40 requirement-to-test-case links / 15 cases)
Highest-requirement test cases	TC-SIM-02 and TC-SEC-01 (5 requirements each)

Analysis. Forward and reverse traceability are complete: all 34 requirements map to at least one of the 15 test cases, and all test cases map to requirements. Thirty-three requirements are accepted; SCM-NF-004 remains partially validated because macOS was not tested. Rishi's final System Validation Report was merged in PR #89 and records E1–E6: E1–E4 exercise SCM-F-043 (CLI invocation), E5 exercises SCM-F-034 (run summary), and E6 exercises SCM-F-044 (daily metrics CSV export). These are results recorded in the report; the manual E1–E6 runs were not independently repeated during this audit.

14. Test Metrics and Reporting

14.1 Quality Metrics and Target Thresholds

The Software Engineering testing process monitors five key quality indicators to evaluate product release readiness:

Metric Category	Specific Indicator	Project Target	Final evidence status (09-11-2026)	Compliance Status
Execution Rate	Test-case specifications with recorded execution results	100%	15 / 15 have recorded results; TC-COV-01 is partial because macOS was not tested	**Recorded; portability partial**
Pass Rate	Percentage of executed automated tests passing	100%	100 / 100 automated tests passed	**Met**
Code Coverage	Statement and branch coverage measured by `pytest-cov`	$\geq 90\%$ (SCM-NF-003)	99.79% line, 99.17% branch; combined 99.66%	**Met**
CI Gate	Automated suite and configured coverage threshold on PRs and pushes to `main`	Required	Workflow runs pytest with branch coverage and `--cov-fail-under=90`	**Configured; no run count asserted here**

14.2 Code Coverage Breakdown by Subsystem Module

The line-coverage counts in the final report are detailed below. The exact combined line-and-branch percentage is reported in §14.4.

Module Under Test	Statements	Missed Statements	Statement Coverage (%)	Module Owner
`src/scm_sim/__init__.py`	1	0	**100%**	Shared
`src/scm_sim/__main__.py`	33	1	**96.97%**	Rohith Karning
`src/scm_sim/analytics.py`	67	0	**100%**	Pranav Achar
`src/scm_sim/inventory.py`	59	0	**100%**	Rohith Karning
`src/scm_sim/orders.py`	66	0	**100%**	Rohanth Ganesha C
`src/scm_sim/shipments.py`	88	0	**100%**	Rishi J Mule
`src/scm_sim/simulator.py`	155	0	**100%**	Rohith Karning
Project Total	**469**	**1**	**99.79%**	**Team 02**

14.3 Test Reporting Protocol

1. **Pull Request Status Checks:** Every pull request targeting `main` executes the GitHub Actions `test` job. The job reports pass/fail status, writes line and branch coverage percentages to the Actions step summary, and uploads JUnit XML plus HTML/XML coverage artifacts.
2. **Defect Tracking via GitHub Issues:** Any failing test or edge-case regression is logged as a GitHub Issue labeled with the affected subsystem (`inventory`, `orders`, `shipments`, `analytics`, `simulator`) and assigned to the module owner.
3. **Formal System Validation Report:** Rishi's final E1–E6 report was merged in PR #89. Its evidence is recorded in `docs/test-reports/system-validation/system-validation-report.md`; the requirement matrix maps E5 to SCM-F-034, E6 to SCM-F-044, and SCM-F-043 to CLI invocation and exit behavior. This Test Plan records the report's existence and mapping without independently claiming that its manual E1–E6 runs were reproduced here.

14.4 Final Metrics (09-11-2026)

Metric	Target	Final Value	Evidence
Automated Tests Executed	100%	100 / 100	Unit Test Report, commit `30702f8`; System Validation Report, commit `b87fd55`
Automated Tests Passed	100%	100 / 100 (0 failed, 0 skipped, 0 errors)	Unit Test Report, commit `30702f8`; System Validation Report, commit `b87fd55`
Statement Coverage	$\geq 90\%$ (SCM-NF-003)	**99.79%** (468 / 469 statements; one missed entrypoint statement)	Code Coverage Report, commit `f4dc0e8`
Branch Coverage	Reported	**99.17%** (119 / 120 branches; one partial)	Branch Coverage Report, commit `f4dc0e8`
Combined line + branch coverage	$\geq 90\%$ (SCM-NF-003)	**99.66%** (587 / 589 measured opportunities)	Code and Branch Coverage Reports, commit `f4dc0e8`
Test Cases with Results	15 / 15 documented	15 / 15 have a recorded result; 14 passed and TC-COV-01 is partial because	Test Plan case records and this metrics report

Metric	Target	Final Value	Evidence
		macOS was not tested	
Manual Test Cases	Results recorded	TC-PERF-01, TC-REP-01 and TC-SEC-02 passed; TC-COV-01 passed on Windows/Linux, macOS not run	System Validation Report, commit `b87fd55`; individual result records
Final system validation E1–E6	PR #89 / issue #65	Rishi's report is merged and records E1–E6; SCM-F-034, SCM-F-043 and SCM-F-044 are mapped separately	System Validation Report, commit `b87fd55`; manual runs not independently repeated for this metrics table

Evidence provenance: the unit-test report identifies commit `30702f8`, both coverage reports identify `f4dc0e8`, and the System Validation Report identifies `b87fd55`. Their reported test totals and coverage figures are consistent (the System Validation Report rounds coverage to whole percentages), but they are not evidence from a single common commit or identical pytest versions.

14.5 Formal Reports and Deliverables

The available test evidence and verification reports are documented in `docs/test-reports/`: 1. Unit Test Report: Execution summary of all 100 unit, integration, and CLI tests. 2. Code Coverage Report: Statement-level line coverage breakdown and missed-line analysis. 3. Branch Coverage Report: Decision-level branch execution analysis. 4. Final System Validation Report v1.0: Rishi's final E1–E6 report, merged in PR #89; the report records E5 for SCM-F-034, E6 for SCM-F-044, and E1–E4 for SCM-F-043.

15. Approvals

By signing below, the approvers confirm that they have read this Test Plan (version 1.0) and its 15 test cases, and agree that:

1. The test items (§2), features to be tested (§3) and features not to be tested (§4) correctly reflect the Supply Chain Management Simulator as built at commit `b3702ef`.
2. The test approach, entry and exit criteria (§5), including security validation (§5.1), are adequate for verifying the requirements in SRS v1.0.
3. The schedule (§7), roles (§9), and suspension and resumption criteria (§12) are accepted by the team.
4. Every requirement ID in the SRS is traced to at least one test case (§13), and the test cases use the IDs in `docs/requirements-registry.md`.

Role	Name	Signature	Date	Decision
Product Owner	Rohith Karning			☐Approved ☐ Approved with changes ☐Rejected
QA Lead	Rohanth Ganesha C			☐Approved ☐ Approved with changes ☐Rejected
Dev Lead	Pranav Achar			☐Approved ☐ Approved with changes ☐Rejected

Approval conditions

- Approval of version 1.0 authorises execution of the manual test cases (TC-PERF-01, TC-SEC-02, and the manual parts of TC-COV-01 and TC-REP-01) according to the schedule in §7.
- Any change to scope, exit criteria or requirement IDs after approval requires a new version of this plan, a new row in the revision history, and re-approval by the same three roles.

- The test summary report (milestone M5, §7) is approved by the same roles once the exit criteria in §5 are met.

Comments from approvers

Approver	Comment	Resolution

Field	Content
TC ID	TC-ANA-01
Title	Delayed-Shipment Detection and Inventory Exception Classification
Requirement(s)	SCM-F-030, SCM-F-031
Type	Functional
Objective	Verify that `find_delayed_shipments` reports only shipments that are genuinely late, computes `days_late` from the correct reference day, and orders the worst delays first; and that `find_stockouts`, `find_low_stock` and `find_excess` partition the stock snapshot correctly at their exact boundary values, using each item's own stock policy thresholds with 20/500 as the global fallback.
Preconditions	The `scm_sim` package is installed (`pip install -e .`) or importable via PYTHONPATH=`src`. No simulation, `Inventory`, `ShipmentManager` or `OrderManager` instance is required for steps 1–7: the Analytics functions read plain data and hold no state. Step 8 uses a `Simulation`.
Test data	Shipments built as stand-ins exposing `shipment_id`, `sku`, `warehouse_id`, `expected_day`, `actual_day`. Stock snapshot with items having custom `StockPolicy` (e.g. `reorder_point=50, max_level=200`, `reorder_point=10, max_level=30`) and items without policies falling back to `reorder_point=20` and `max_level=500`.
Steps	1. Call `find_delayed_shipments([shipment], current_day=8)` for the shipment expected on day 5 and still in transit; inspect `days_late`.2. Call `find_delayed_shipments([shipment], current_day=20)` for the same shipment delivered on day 7; inspect `days_late`.3. Call `find_delayed_shipments` on the on-time (day 5) and early (day 3) shipments at `current_day = 10`.4. Call `find_delayed_shipments` on two shipments 1 and 4 days late at `current_day = 10`; inspect the order of `shipment_id` in the result.5. Call `find_stockouts(stock)`, `find_low_stock(stock)` and `find_excess(stock)` on the stock snapshot with default thresholds; compare each result against the expected tuples.6. Call `find_low_stock(stock, reorder_point=21)` and `find_excess(stock,`

Field	Content
	`max_level=100`) to confirm the thresholds are applied per call.7. Call `find_low_stock` and `find_excess` on items with custom policies; verify that each item's own `reorder_point` and `max_level` determine alerts.8. Verify in a `Simulation` that daily report alerts evaluate using per-item policy thresholds.
Expected result	1. `days_late = 3` for the in-transit shipment, computed as `current_day 8 – expected_day 5`.2. `days_late = 2`, computed from the recorded `actual_day 7` and **not** from `current_day 20`.3. An empty list: neither shipment is late.4. The result is ordered worst first.5. Boundary values 0, 20, 500 evaluate strictly per specification.6. Custom thresholds override defaults per function call.7. Items with individual stock policies alert on their specific thresholds; items without policy use 20 and 500.8. Simulation daily reports reflect per-item thresholds accurately.
Pass/fail criteria	**Pass** if all listed pytest tests pass, lateness reference days are respected, ordering is descending, stock classifiers produce exact boundary results, and per-item policies govern alerts with 20/500 fallback; **Fail** if timing calculations fail, lists are unsorted, or policy thresholds are ignored.
Automated by	`tests/test_analytics.py::test_in_transit_late_uses_current_daytests/test_analytics.py::test_delivered_late_uses_actual_day_not_current_daytests/test_analytics.py::test_on_time_and_early_are_excludedtests/test_analytics.py::test_in_transit_not_yet_due_is_excludedtests/test_analytics.py::test_arrival_on_day_zero_is_not_treated_as_in_transittests/test_analytics.py::test_sorted_by_days_late_descendingtests/test_analytics.py::test_stockouts_only_exact_zero_and_sortedtests/test_analytics.py::test_low_stock_boundaries_zero_excluded_and_20_includedtests/test_analytics.py::test_low_stock_respects_custom_reorder_pointtests/test_analytics.py::test_excess_boundaries_500_excluded_and_501_includedtests/test_analytics.py::test_excess_surplus_with_custom_max_leveltests/test_analytics.py::test_empty_stock_gives_empty_resultstests/test_analytics.py::test_low_stock_and_excess_use_each_items_own_policytests/test_analytics.py::test_items_without_policy_fall_back_to_global_thresholdstests/test_analytics.py::test_build_report_passes_policies_throughtests/test_simulator.py::test_report_uses_per_item_policy_th

Field	Content
	`resholds`
Actual result	Pass. Final automated suite: 100 tests passed. See final unit-test report.
Status	Passed

Updated 08-11-2026: low stock and excess now use each item's own stock policy with 20/500 fallback (#71) (#61); final suite evidence reconciled 09-11-2026.

Field	Content
TC ID	TC-ANA-02
Title	Order Metrics, Fill Rate, Daily Report Composition, Formatting, and Run Summary
Requirement(s)	SCM-F-032, SCM-F-033, SCM-F-034
Type	Functional
Objective	Verify that `order_metrics` counts orders per status and computes the fill rate accurately, that `build_report` and `format_report` produce complete daily reports with "No issues" for empty sections, and that `summarize_run` and `format_run_summary` aggregate metrics across simulation history.
Preconditions	The `scm_sim` package is installed or importable.
Test data	Order lists with various statuses (PENDING, FULFILLED, BACKORDERED, CANCELLED, unknown strings); daily report fixtures; multi-day report histories.
Steps	1. Call `order_metrics` on orders with standard statuses.2. Call `order_metrics` excluding cancelled orders from fill rate denominator.3. Call `order_metrics` on an empty list.4. Call `order_metrics` mixing `OrderStatus` members and strings.5. Call `build_report` and inspect all keys and contents.6. Call `build_report` forwarding custom thresholds.7. Call `format_report` and inspect headers and counts.8. Call `format_report` on empty reports and verify "No issues" sections.9. Call `summarize_run(history)` on a multi-day report history and verify days, delays, stockouts, low stock, excess, peak delayed, and final fill rate.10. Call `summarize_run([])` on empty history.11. Call `format_run_summary(summary)` and verify `=== Run Summary ===` and field lines.12. Call `order_metrics` with unknown status strings (e.g. "lost") and verify total count handling.
Expected result	Status counts and fill rate calculate correctly; cancelled orders are excluded from denominator; empty list gives 0.0 fill rate without ZeroDivisionError; `build_report` produces all guaranteed keys; `format_report` formats headers and "No issues"; `summarize_run` correctly aggregates multi-day run metrics; empty history evaluates cleanly with zeroes; `format_run_summary` outputs header and all summary lines; unknown statuses are counted in total

Field	Content
	without corrupting status counts.
Pass/fail criteria	**Pass** if all listed pytest tests pass, fill rate calculations match formulas, reports are complete, and run summaries aggregate accurately; **Fail** if math errors occur, keys are missing, or summary calculations are incorrect.
Automated by	`tests/test_analytics.py::test_order_metrics_counts_with_string_statusestests/test_analytics.py::test_order_metrics_accepts_enum_statusestests/test_analytics.py::test_enum_and_string_statuses_can_be_mixedtests/test_analytics.py::test_fill_rate_excludes_cancelled_orderstests/test_analytics.py::test_fill_rate_is_zero_with_no_orderstests/test_analytics.py::test_fill_rate_is_zero_when_all_cancelledtests/test_analytics.py::test_build_report_has_all_keystests/test_analytics.py::test_build_report_contentstests/test_analytics.py::test_build_report_passes_custom_thresholds_throughtests/test_analytics.py::test_format_report_has_section_headers_and_countstests/test_analytics.py::test_format_report_prints_no_issues_for_empty_sectionstests/test_analytics.py::test_summarize_run_and_formattests/test_analytics.py::test_summarize_run_empty_historytests/test_analytics.py::test_order_metrics_unknown_status`
Actual result	Pass. Final automated suite: 100 tests passed; run-summary tests pass. See final unit-test report.
Status	Passed

Updated 08-11-2026: added run-summary coverage for SCM-F-034 (#79) (#61); final suite evidence reconciled 09-11-2026.

Field	Content
TC ID	TC-COV-01
Title	Test Coverage Gate and Cross-Platform Portability
Requirement(s)	SCM-NF-003, SCM-NF-004
Type	Maintainability
Objective	Verify that the automated suite passes and maintains at least 90% line coverage of `scm_sim` as required by SCM-NF-003, that the configured CI coverage gate passes, and that the simulator runs on Windows, Linux and macOS on Python 3.11 or later using only the standard library at runtime. Branch coverage is also measured and reported.
Preconditions	*Automated part:* the repository is hosted on GitHub and the GitHub Actions `test` job is enabled for the `main` branch and for pull requests. *Manual part:* a

Field	Content
	Windows 11 machine with Python 3.11 and a Linux machine with Python 3.11, each with `pytest` and `pytest-cov` installed from `requirements.txt`.
Test data	The complete test suite across all test files; standard demo scenario `build_demo(seed=42)` for manual runs. `requirements.txt` inspected for dependencies.
Steps	*Automated part.* 1. Push a commit to a feature branch and open a pull request to `main`.2. Wait for the `test` job on `ubuntu-latest` with Python 3.11 to complete.3. Inspect the job summary and coverage reports for `scm_sim`.4. Confirm that merging is blocked if tests or the 90% coverage threshold fails.*Manual part.* 5. On Windows 11, install dependencies and run `pytest --cov=scm_sim --cov-branch`.6. On Windows 11, execute `scm-sim --days 10 --seed 42`.7. Repeat steps 5 and 6 on a Linux environment.8. Inspect `requirements.txt` and `pyproject.toml` to verify zero runtime dependencies.9. Run the simulator in an environment containing only standard library packages.10. Record host OS and Python version in test records.
Expected result	CI job passes with 0 failures and the configured `--cov-fail-under=90` gate; PR merge is blocked on failure; test suite and CLI execute cleanly on each tested platform; runtime dependencies are zero.
Pass/fail criteria	SCM-NF-003 passes when the tests pass and line coverage is at least 90%; report branch coverage and CI's configured combined coverage gate separately. SCM-NF-004 is fully passed only after Windows, Linux and macOS testing; without macOS evidence, mark it partial.
Automated by	GitHub Actions `test` job in `.github/workflows/ci.yml`, running `pytest -v --cov=scm_sim --cov-branch --cov-fail-under=90` on `ubuntu-latest` with Python 3.11 on every push to `main` and every pull request.
Actual result	100 automated tests passed; line coverage 99.79%, branch coverage 99.17%, combined coverage 99.66%. Windows 11, Linux and CI Ubuntu evidence is recorded; macOS was not tested. See unit-test report, line-coverage report and branch-coverage report.
Status	SCM-NF-003 Passed; SCM-NF-004 Partially validated (macOS not tested)

Updated 09-11-2026: reconciled 100-test and exact coverage evidence; explicitly retained SCM-NF-004 as partial because macOS was not tested.

TC-INV-01: Stock Tracking and Warehouse Capacity

Field	Content
TC ID	TC-INV-01
Title	Add and remove stock, reject over-removal and over-

Field	Content
	capacity additions, unlimited capacity for unregistered warehouses
Requirement(s)	SCM-F-001, SCM-F-002 (also supports SCM-SR-001, SCM-SR-002)
Type	Functional (unit, boundary-value)
Preconditions	1. Python 3.11 virtual environment with `pip install -r requirements.txt` completed. 2. Repository at commit `b3702ef` (or a later green `main`). 3. Each step starts from a new, empty `Inventory()` from `scm_sim.inventory`.
Test data	Made-up IDs as used in the test code: warehouses `WH1`, `WH2`; SKUs `SKU1`, `SKU2` (equivalent to `WH-NORTH` / `WIDGET` in the demo). Quantities: 50, 20, 5, 10, 60, 40, 41, 1 000 000. Capacities: 100 for `WH1` in steps 4–5, 10 for `WH1` in step 6.
Objective	Prove that the Inventory records stock correctly, never lets stock fall below 0, and never lets a registered warehouse exceed its capacity, while unregistered warehouses are unlimited.
Steps	1. `add("WH1", "SKU1", 50)`. 2. With 50 in stock, `remove("WH1", "SKU1", 20)`. 3. With 5 in stock, `remove("WH1", "SKU1", 10)`. 4. Call `remove("WH1", "SKU1", 0)` and `remove("WH1", "SKU1", -1)`. 5. `register_warehouse("WH1", 100)`; `add("WH1", "SKU1", 60)`; `add("WH1", "SKU2", 40)`; read `total_in_warehouse("WH1")`. 6. `register_warehouse("WH1", 100)`; `add("WH1", "SKU1", 60)`; `add("WH1", "SKU2", 41)`; read `level` of both SKUs and `total_in_warehouse("WH1")`. 7. `register_warehouse("WH1", 10)`; `add("WH2", "SKU1", 1_000_000)`.
Expected result	1. Returns 50. 2. Returns 30. 3. Raises `InsufficientStockError`. 4. Both calls raise `ValueError("qty must be positive")`. 5. The second add returns 40; the warehouse total is exactly 100 (adding up to capacity is allowed). 6. The second add raises `CapacityExceededError`; afterwards `level("WH1", "SKU1") == 60`, `level("WH1", "SKU2") == 0` and the total is 60 (stock unchanged). 7. Returns 1 000 000 (the unregistered `WH2` has no limit).
Pass/fail criteria	**Pass** if every step gives exactly the expected return value or exception and all pytest tests listed below pass. **Fail** if any value differs, the wrong exception (or none) is raised, or stock changes after a rejected call.
Automated by	`tests/test_inventory.py::test_add_and_level`, `tests/test_inventory.py::test_remove_reduces_stock`,

Field	Content
	`tests/test_inventory.py::test_remove_too_much_raises`, `tests/test_inventory.py::test_remove_rejects_non_positive_quantity`, `tests/test_inventory.py::test_add_up_to_exact_capacity_is_allowed`, `tests/test_inventory.py::test_add_beyond_capacity_raises_and_leaves_stock_unchanged`, `tests/test_inventory.py::test_unregistered_warehouse_has_unlimited_capacity`. Supporting: `test_stockout_detected`, `test_register_warehouse_rejects_non_positive_capacity`, `test_total_in_warehouse_sums_all_skus`, `test_existing_behaviour_unchanged_with_capacity_registered`
Actual result	Pass. Final automated suite: 100 tests passed. See final unit-test report.
Status	Passed

Revision 08-11-2026: Added automated coverage for zero and negative removal quantities, aligned warehouse capacity steps with the current implementation, and removed the obsolete coverage gap note.

Revision 09-11-2026: Reconciled final automated-suite evidence to the unit-test report and aligned functional/security scope with the traceability maps.

TC-INV-02: Stock Policies, Reorder Suggestions and Snapshots

Field	Content
TC ID	TC-INV-02
Title	Validate stock policies, detect reorder and excess at their boundaries, list reorder suggestions in order, and return independent snapshots
Requirement(s)	SCM-F-003, SCM-F-004 (also supports SCM-SR-001, SCM-SR-007)
Type	Functional (unit, boundary-value)
Preconditions	1. Python 3.11 virtual environment with `pip install -r requirements.txt` completed. 2. Repository at commit `b3702ef` (or a later green `main`). 3. Each step starts from a new, empty `Inventory()`; `StockPolicy` imported from `scm_sim.inventory`.
Test data	Made-up IDs as used in the test code: warehouses `WH1`, `WH2`, `WH3`; SKUs `SKU1`, `SKU2`, `SKU3`, `SKU9`. Invalid policies: `reorder_point=-1`, `reorder_quantity=-1`, `max_level=-1`, and `reorder_point=50, max_level=49`. Boundary levels: 21 → 20 for reorder point 20; 100 → 101 for max level 100.
Objective	Prove that invalid policies are rejected, that reorder and excess decisions switch exactly at their thresholds, that reorder suggestions include every tracked item that needs one in a fixed sorted order, and that a

Field	Content
	snapshot cannot be used to change stock.
Steps	1. Create `StockPolicy(**kwargs)` for each of the four invalid parameter sets. 2. Set `StockPolicy(reorder_point=20)` on (`WH1`, `SKU1`), add 21 units and call `needs_reorder`; remove 1 unit and call `needs_reorder` again. 3. Set `StockPolicy(reorder_point=0, max_level=100)` on (`WH1`, `SKU1`), add 100 units and call `is_excess`; add 1 more and call `is_excess` again. 4. Add 5 × (`WH2`, `SKU1`), 300 × (`WH1`, `SKU9`), 10 × (`WH1`, `SKU2`) then remove those 10, and set `StockPolicy(reorder_quantity=40)` on (`WH1`, `SKU1`) without stock; call `reorder_suggestions()`. 5. Add 10 × (`WH1`, `SKU1`) and 5 × (`WH2`, `SKU2`); take `snapshot()`; set its (`WH1`, `SKU1`) entry to 999 and add a key (`WH3`, `SKU3`); read `level` and take a new `snapshot()`.
Expected result	1. Each of the four raises `ValueError`. 2. `False` at 21, `True` at 20 (the reorder point itself triggers a reorder). 3. `False` at 100, `True` at 101 (only levels above max level are excess). 4. Exactly `[("WH1", "SKU1", 40), ("WH1", "SKU2", 100), ("WH2", "SKU1", 100)]`: the policy-only item and the sold-out item are included, the healthy `SKU9` is not, and the list is sorted by warehouse then SKU. 5. The first snapshot equals `{("WH1", "SKU1"): 10, ("WH2", "SKU2"): 5}`; after editing it, `level("WH1", "SKU1") == 10`, `level("WH3", "SKU3") == 0`, and a new snapshot still equals the original values.
Pass/fail criteria	**Pass** if every step gives exactly the expected result and all pytest tests listed below pass. **Fail** if an invalid policy is accepted, a boundary decision is off by one, the suggestion list differs in content or order, or editing a snapshot changes the inventory.
Automated by	`tests/test_inventory.py::test_stock_policy_rejects_invalid_values` (4 parametrised cases), `tests/test_inventory.py::test_needs_reorder_at_boundary`, `tests/test_inventory.py::test_is_excess_only_above_max_level`, `tests/test_inventory.py::test_reorder_suggestions_sorted_and_only_tracked_items`, `tests/test_inventory.py::test_snapshot_returns_independent_copy`. Supporting: `test_stock_policy_defaults` (20 / 100 / 500), `test_stock_policy_allows_max_level_equal_to_reorder_point`, `test_get_policy_returns_default_when_unset`, `test_set_policy_is_per_warehouse_and_s

Field	Content
	ku, test_reorder_suggestions_empty_for_new_inventory
Actual result	Pass. Final automated suite: 100 tests passed. See final unit-test report.
Status	Passed

Revision 08-11-2026: Confirmed the documented stock-policy boundaries and reorder suggestion order against the current implementation and automated tests.

Revision 09-11-2026: Reconciled final automated-suite evidence to the unit-test report and aligned functional/security scope with the traceability maps.

Test Case Specification: TC-ORD-01

Field	Content
TC ID	TC-ORD-01
Title	Order Creation, Sequential Identification, and Inventory Fulfillment Validation
Requirement(s)	SCM-F-010, SCM-F-011
Type	Functional
Preconditions	Initialized Inventory instance; OrderManager bound to the inventory; warehouse WH-NORTH has either sufficient or deficient stock of SKU WIDGET.
Test data	Customer: C-ACME, Warehouse: WH-NORTH, SKU: WIDGET, Quantities: 10, 5, 0, -5, Day: 1
Objective	Verify sequential ID generation, negative/zero quantity rejection without sequence gap, atomic inventory deduction upon fulfillment, and backorder transition on stock shortage.
Steps	1. Call mgr.create("C-ACME", "WH-NORTH", "WIDGET", 10, day=1) and mgr.create("C-ACME", "WH-NORTH", "WIDGET", 5, day=1).2. Attempt to create an order with quantity 0.3. Add 20 units of WIDGET to WH-NORTH, create order for 10 units, call try_fulfil(order, day=1).4. Reset stock to 5 units, create order for 10 units, call try_fulfil(order, day=1).5. Call create with quantity 0 and −5 (each raises ValueError), then create two valid orders and verify their allocated IDs.6. Construct an Order dataclass directly with quantity 0.
Expected result	1. Orders receive sequential IDs ORD-0001 and ORD-0002 with status PENDING.2. Creation with quantity 0 raises ValueError.3. When stock is 20, try_fulfil returns True, status becomes FULFILLED, fulfilled_day=1, stock decreases to 10.4. When stock is 5, try_fulfil returns False, status becomes BACKORDERED, stock remains 5 without partial fulfillment.5. Both invalid calls raise ValueError; subsequent valid orders receive ORD-0001 and ORD-0002 — rejected calls do not consume sequence IDs.6. Direct instantiation with

Field	Content
	non-positive quantity raises `ValueError` in `Order.__post_init__`.
Pass/fail criteria	Pass if all returned IDs, exceptions, statuses, and stock levels match expectations; Fail if quantity 0 is accepted, partial fulfillment occurs, or IDs are non-sequential.
Automated by	`tests/test_orders.py::test_create_order_id_sequencetests/test_orders.py::test_invalid_quantitytests/test_orders.py::test_fulfil_enough_stocktests/test_orders.py::test_backorder_when_shorttests/test_orders.py::test_rejected_order_does_not_consume_an_idtests/test_orders.py::test_order_dataclass_rejects_non_positive_quantity_directly`
Actual result	Pass. Final automated suite: 100 tests passed. See final unit-test report.
Status	Passed

Updated 08-11-2026: Invalid quantity no longer consumes an order ID (#68); added steps 5–6, `test_rejected_order_does_not_consume_an_id`, and `test_order_dataclass_rejects_non_positive_quantity_directly` to Automated by (#59). Final suite evidence reconciled 09-11-2026.

Test Case Specification: TC-ORD-02

Field	Content
TC ID	TC-ORD-02
Title	FIFO Backorder Reprocessing, Fulfillment Priority, and Order Cancellation Rules
Requirement(s)	SCM-F-012, SCM-F-013
Type	Functional
Preconditions	Initialized `OrderManager` and `Inventory`; orders in `BACKORDERED` or `PENDING` states; inventory restocked or exhausted.
Test data	Customers: `C-ACME`, `C-BETA`; Warehouse: `WH-NORTH`; SKU: `WIDGET`; Quantities: `10`, `10`; Days: `1`, `2`, `3`
Objective	Verify that `process_open()` prioritizes older orders over newer orders when stock is constrained, existing back-orders are retried before today's new demand, correctly sets status to `FULFILLED`, enforces cancellation guards preventing cancellation of fulfilled orders, and rejects unknown order IDs or reprocessing fulfilled/cancelled orders.
Steps	1. Create order 1 on day 1 (qty 10) and order 2 on day 2 (qty 10). Attempt fulfillment with 0 stock; verify both are `BACKORDERED`.2. Add 15 units of `WIDGET` to warehouse on day 3.3. Call `mgr.process_open(day=3)`.4. Attempt to cancel an active unfulfilled order.5. Attempt to cancel an already fulfilled order.6. Look up an unknown order ID

Field	Content
	ORD-0042 with get.7. Fulfill one order and cancel another, then call try_fulfil on both again on a later day.8. In simulation, retry open back-orders before today's new orders when fresh stock arrives on the same day.
Expected result	1. Both orders marked BACKORDERED.2. When 15 units are available, process_open(day=3) fulfills the older order (Day 1) and leaves the newer order (Day 2) BACKORDERED. Stock reduces to 5 units.3. Cancelling an unfulfilled order succeeds with status CANCELLED.4. Cancelling a FULFILLED order raises ValueError.5. mgr.get("ORD-0042") raises KeyError.6. Calling try_fulfil on already FULFILLED or CANCELLED order returns False and leaves stock unchanged (SCM-SR-003).7. In simulation step, existing back-orders are retried and filled before today's new demand is processed.
Pass/fail criteria	Pass if older orders are prioritized strictly over newer orders during constrained stock replenishment, cancellation rules are enforced, unknown IDs raise KeyError, and re-fulfilling finished orders returns False without stock deduction; Fail if newer order is fulfilled first or fulfilled order is cancelled.
Automated by	tests/test_orders.py::test_backorder_later_fulfilled_by_process_opentests/test_orders.py::test_oldest_first_prioritytests/test_orders.py::test_cancel_rulestests/test_orders.py::test_get_unknown_order_raises_key_errortests/test_orders.py::test_fulfilled_or_cancelled_orders_are_never_reprocessedtests/test_simulator.py::test_older_backorder_is_served_before_new_order_on_same_day
Actual result	Pass. Final automated suite: 100 tests passed. See final unit-test report.
Status	Passed

Updated 08-11-2026: Open back-orders are now retried before today's new orders (#69); updated steps and expected results, and added test_get_unknown_order_raises_key_error, test_fulfilled_or_cancelled_orders_are_never_reprocessed, and test_older_backorder_is_served_before_new_order_on_same_day to Automated by (#59). Final suite evidence reconciled 09-11-2026.

Test Case Specification: TC-PERF-01

Field	Content
TC ID	TC-PERF-01
Title	System Execution Performance Benchmark for Long-Horizon and CLI Demo Runs
Requirement(s)	SCM-NF-001
Type	Performance
Preconditions	Python 3.11 installed; clean operating system state on developer laptop; terminal session open.

Field	Content
Test data	365-day programmatic scenario (`build_demo(42).run(365)`) and 10-day CLI scenario (`scm-sim --days 10 --seed 42`).
Objective	Verify that a full 365-day simulation run executes in ≤ 1.0 second and the 10-day CLI invocation (including interpreter startup and formatting) executes in ≤ 1.0 second across 3 trials.
Steps	1. Execute `python -c "import time; from scm_sim.simulator import build_demo; s=build_demo(42); t=time.perf_counter(); s.run(365); print(round(time.perf_counter()-t,3),'s')"` 3 times.2. Execute `Measure-Command { scm-sim --days 10 --seed 42 } | Select-Object TotalSeconds` (or `time scm-sim --days 10 --seed 42` on Linux/macOS) 3 times.3. Record the execution time for each iteration and verify all trials remain under 1.0 second.
Expected result	1. 365-day programmatic run finishes in < 1.0 s (measured baseline: ~0.014–0.027 s).2. 10-day CLI run finishes in < 1.0 s (measured baseline: ~0.20–0.26 s).
Pass/fail criteria	Pass if all trial execution times remain ≤ 1.0 second; Fail if any execution exceeds 1.0 second.
Automated by	Manual benchmark execution using Python `time.perf_counter()` and terminal `Measure-Command` / `time`.
Actual result	Pass (10-day CLI 0.20-0.26 s, 365-day run 0.014-0.027 s; evidence docs/testplan/results/TC-PERF-01.md)
Status	Passed

Updated 08-11-2026: Actual result = Pass, Status = Passed, evidence docs/testplan/results/TC-PERF-01.md (10-day CLI 0.20-0.26 s, 365-day run 0.014-0.027 s) (#59).

Field	Content
TC ID	TC-REP-01
Title	Seeded Reproducibility of Daily Reports and CLI Output
Requirement(s)	SCM-NF-002
Type	Reliability
Objective	Verify that supplying the same random seed with the same scenario produces byte-for-byte identical daily reports and identical console output on every run, and that a different seed changes the simulated delays deterministically — so that any reported failure can be reproduced exactly from its seed.
Preconditions	The `scm_sim` package is installed (`pip install -e .`) and the demo scenario is available through `build_demo(seed)`. For the manual part, commands are run as `scm-sim --days 10 --seed 42` and `scm-sim --days 10 --seed 1`.
Test data	Demo scenario built by `build_demo(seed=42)`:

Field	Content
	suppliers `SUP-RELIABLE` (reliability 1.0) and `SUP-FLAKY` (reliability 0.3); warehouses `WH-NORTH` (capacity 1200) and `WH-SOUTH` (capacity 500); SKUs `WIDGET`, `GADGET`, `GIZMO`; 14 customer orders including customer `C-ACME`. Seed 42 for the identity check; seed 1 for the variation check.
Steps	*Automated part.* 1. Build the demo with `build_demo(seed=42)` and call `run(10)`; keep the list of daily reports.2. Build a second, independent demo with `build_demo(seed=42)` and call `run(10)`.3. Compare the two lists of reports day by day.*Manual part.* 4. Run `scm-sim --days 10 --seed 42` twice, redirecting each run's full output to a separate file.5. Compare the two output files with `fc` / `diff`.6. Run `scm-sim --days 10 --seed 1` and compare the delayed-shipment lines against the seed-42 output.7. Record the host operating system and Python version used for the manual runs.
Expected result	1. Ten reports are produced, one per simulated day.2. Ten reports are produced from the second, independent instance.3. The two lists are equal: every report for day *n* is identical in every key.4. Each run prints a day-by-day log and a final report and exits with code 0.5. The two files are identical (`fc` / `diff` reports no differences).6. The seed-1 run produces different shipment delays from the seed-42 run, and the difference is deterministic.7. The host details are recorded in the test report.
Pass/fail criteria	**Pass** if the automated test passes and both seed-42 manual runs produce identical output while the seed-1 run produces repeatable variations; **Fail** if any values differ between identical seeds or match across different seeds.
Automated by	`tests/test_simulator.py::test_same_seed_gives_identical_reports`
Actual result	Pass; automated report equality and manual CLI comparisons passed. Final automated suite: 100 tests passed. See reproducibility results and final unit-test report.
Status	Passed

Updated 08-11-2026: recorded manual test results and updated status to Passed (#61); final suite evidence reconciled 09-11-2026.

Field	Content
TC ID	TC-SEC-01
Title	Public Input Validation, Invariant Enforcement, and State Tamper Resistance
Requirement(s)	SCM-SR-001, SCM-SR-002, SCM-SR-003, SCM-SR-004, SCM-SR-007
Type	Security
Objective	Verify that domain operations reject invalid boundary

Field	Content
	parameters before mutating state, enforce warehouse capacity and non-negative stock invariants, guard order lifecycles against invalid status changes, reject unknown IDs with `KeyError`, and return immutable independent snapshots.
Preconditions	Initialized `Inventory`, `OrderManager`, `ShipmentManager`, and `Simulation` instances.
Test data	Negative quantities (-10, 0); negative lead times (-1); invalid reliabilities (-0.1, 1.5); non-existent entity IDs (`SUP-999`, `ORD-999`, `WH-999`); warehouse capacity 100 with stock addition of 101.
Steps	1. Attempt adding and removing non-positive quantities in Inventory, Orders, and Shipments.2. Attempt removing more units than available in warehouse stock.3. Attempt adding units exceeding registered warehouse capacity.4. Attempt cancelling a FULFILLED order and re-fulfilling a CANCELLED order.5. Attempt dispatching and scheduling orders referencing unknown supplier and warehouse IDs.6. Mutate the dictionary returned by `Inventory.snapshot()` and verify internal `_stock`.
Expected result	`ValueError` raised for invalid parameters with state left unchanged; `InsufficientStockError` and `CapacityExceededError` raised with inventory intact; `cancel()` on FULFILLED raises `ValueError`; unknown entity references raise `KeyError`; mutating returned snapshot has zero effect on internal inventory.
Pass/fail criteria	**Pass** if all invalid input vectors are rejected, state invariants remain uncompromised, and snapshots are independent; **Fail** if state corruption, silent mutation, or unhandled crashes occur.
Automated by	`tests/test_inventory.py::test_remove_rejects_non_positive_quantitytests/test_inventory.py::test_remove_too_much_raisestests/test_inventory.py::test_register_warehouse_rejects_non_positive_capacitytests/test_inventory.py::test_add_beyond_capacity_raises_and_leaves_stock_unchangedtests/test_inventory.py::test_stock_policy_rejects_invalid_valuestests/test_inventory.py::test_snapshot_returns_independent_copytests/test_orders.py::test_invalid_quantitytests/test_orders.py::test_rejected_order_does_not_consume_an_idtests/test_orders.py::test_order_dataclass_rejects_non_positive_quantity_directlytests/test_orders.py::test_get_unknown_order_raises_key_errortests/test_orders.py::test_cancel_rulestests/test_orders.py::test_fulfilled_or_cancelled_orders_are_neve`

Field	Content
	`r_reprocessedtests/test_shipments.py::test_supplier_validationtests/test_shipments.py::test_add_supplier_and_reject_duplicate_and_unknowntests/test_shipments.py::test_reject_non_positive_quantitytests/test_simulator.py::test_schedule_order_validates_inputtests/test_simulator.py::test_set_policy_rejects_unknown_supplier`
Actual result	Pass. Final automated suite: 100 tests passed. See final unit-test report.
Status	Passed

Reviewed 08-11-2026: no change needed after Sprint 2. Updated 08-11-2026: verified all security and input validation tests pass on current main (#62); final suite evidence reconciled 09-11-2026.

Revision 09-11-2026: Expanded the automated-test mapping for invalid inputs, unknown order IDs and closed-order protection; all names verified against `tests/`.

TC-SEC-02: Branch Protection, Secrets and Dependency Check

Field	Content
TC ID	TC-SEC-02
Title	`main` accepts only reviewed, CI-tested changes; the repository holds no secrets; the simulator has no network access and no runtime third-party dependencies
Requirement(s)	SCM-SR-005, SCM-SR-006 (security objective SCM-SO-2)
Type	Security (process)
Preconditions	1. Tester is logged in to GitHub as the repository owner (`rohithkarning-debug`), who has admin rights, so that step 2 also checks that the rules apply to admins. 2. A local clone with `git checkout main && git pull origin main` done and a clean working tree. 3. A second team member is available for step 3. 4. The steps are run in Git Bash (or a Linux/macOS shell) from the repository root.
Test data	Throwaway local branch `sec-check`; one **empty** commit `test: direct push check (TC-SEC-02)`. Secret patterns: GitHub tokens (`ghp_…`, `github_pat_…`), AWS keys (`AKIA…`), private-key headers, and assignments to `api_key`, `secret`, `password`, `passwd`, `token`. No real credentials are used.
Objective	Prove that no one, including an admin, can change `main` without a pull request, a non-author approval and a passing `test` check, and that the code base exposes no secrets, network access or runtime dependencies.
Steps	1. **Settings review:** open Settings → Branches → the rule for `main` and record each setting. 2. **Direct push (as admin):** `git checkout -b sec-check`; `git`

Field	Content
	`commit --allow-empty -m "test: direct push check (TC-SEC-02)"`; `git push origin sec-check:main`. Then `git push --force origin sec-check:main`. 3. **Merge without approval:** `git push -u origin sec-check`; open a pull request `sec-check` → `main`; wait for CI; try to merge without any review; try to approve as the author; ask a teammate to confirm the merge button state. Close the PR without merging and delete the remote and local `sec-check` branches. 4. **Secrets scan:** `git log --all -p | grep -nEi "ghp_[A-Za-z0-9]{36}|github_pat_|AKIA[0-9A-Z]{16}|BEGIN (RSA|OPENSSH|EC) PRIVATE KEY|(api[_-]?key|secret|password|passwd|token)\s*[:=]"`; then check Security → Secret scanning alerts on GitHub. 5. **Network use:** `grep -rnE "socket|urllib|http|requests" src/scm_sim/`. 6. **Dependencies:** `cat requirements.txt`; `grep -rhE "^\s*(from|import) " src/scm_sim/*.py | sort -u`.
Expected result	1. The rule shows: *Require a pull request before merging* on, *Required approvals* = 1, *Dismiss stale pull request approvals when new commits are pushed* on, *Require status checks to pass* with required check `test`, *Require branches to be up to date before merging* on, *Do not allow bypassing the above settings* (rules apply to administrators) on, *Allow force pushes* off, *Allow deletions* off. 2. Both pushes are rejected by GitHub with a protected-branch error (e.g. `GH006: Protected branch update failed`), even though the tester is an admin; `main` on GitHub is unchanged. 3. The merge button is disabled with *Review required*; GitHub does not allow the author to approve their own PR; the PR is closed unmerged and the branch deleted. 4. The search prints nothing and GitHub shows no open secret-scanning alerts. 5. No matches: the simulator contains no network code. 6. `requirements.txt` lists only `pytest>=8.0` and `pytest-cov>=5.0` (test tools); `src/scm_sim/` imports only the standard library (`__future__`, `argparse`, `dataclasses`, `enum`, `random`, `typing`) and its own `scm_sim` modules.
Pass/fail criteria	**Pass** if all six expected results are observed. **Fail** if any branch-protection setting differs from step 1, any push in step 2 reaches `main`, a PR can be merged without a non-author approval and a green `test` check, any secret is found, or any network or third-party runtime import exists. If step 2 unexpectedly succeeds, record a High-severity defect and remove the empty commit from `main` through a reviewed PR.
Automated by	Not automated (manual process test). Steps 4–6 are

Field	Content
	read-only shell commands; steps 1–3 need GitHub admin access.
Actual result	Pass - execution evidence: docs/testplan/results/TC-SEC-02.md
Status	Passed

Pre-check (read-only, 11-10-2026; not a substitute for execution). At pre-check time, the public GitHub API reported `main` as `protected: true` with required status check `test` and `enforcement_level: everyone`, and all 20 CI runs up to run #20 succeeded. A dry run of steps 4–6 on commit `b3702ef` produced the expected output. The subsequent full execution on 05-11-2026, including the branch-rule and push/merge checks, is recorded in the linked result report.

Revision 09-11-2026: Clarified that the 11-10 pre-check was superseded by the full execution dated 05-11-2026; linked execution evidence remains authoritative.

Field	Content
TC ID	TC-SHP-01
Title	Supplier Validation, Identifier Sequencing, and Dispatch Lead-Time Computation
Requirement(s)	SCM-F-020, SCM-F-021
Type	Functional
Objective	Verify that supplier registration enforces ID uniqueness and parameter constraints, that shipments generate sequential IDs formatted as `SHP-XXXX`, and that expected arrival days correctly reflect dispatch day plus supplier lead time.
Preconditions	An initialized `Inventory` instance and an instantiated `ShipmentManager`.
Test data	Suppliers `SUP-01` (lead_time_days=2, reliability=1.0), `SUP-02` (lead_time_days=3); warehouse `WH-01`; SKU `SKU-01`; quantity=50; dispatch day=2.
Steps	1. Instantiate `Supplier` with valid and invalid bounds (lead_time < 0, reliability < 0.0 or > 1.0).2. Register `SUP-01` with `ShipmentManager.add_supplier()` and attempt duplicate registration.3. Dispatch a shipment with an unregistered supplier ID `SUP-999`.4. Dispatch two shipments with positive quantities from `SUP-01` and verify generated IDs.5. Dispatch with quantity <= `0` and verify exception handling.6. Dispatch from `SUP-02` (lead time 3) on day 2 and verify `expected_day`.
Expected result	Invalid parameters and duplicate supplier registrations raise `ValueError`; unknown supplier dispatches raise `KeyError`; non-positive quantities raise `ValueError`; generated shipment IDs follow `SHP-0001`, `SHP-0002`; `expected_day` strictly equals `2 + 3 = 5`.
Pass/fail criteria	**Pass** if all assertions hold and exceptions are raised without state mutation; **Fail** if invalid parameters are accepted, IDs collide, or `expected_day` is miscalculated.

Field	Content
Automated by	`tests/test_shipments.py::test_supplier_validationtests/test_shipments.py::test_add_supplier_and_reject_duplicate_and_unknowntests/test_shipments.py::test_expected_day_calculated_from_lead_timetests/test_shipments.py::test_shipment_id_sequence_and_in_transit_filteringtests/test_shipments.py::test_reject_non_positive_quantity`
Actual result	Pass. Final automated suite: 100 tests passed. See final unit-test report.
Status	Passed

Reviewed 08-11-2026: no change needed after Sprint 2. Updated 08-11-2026: verified baseline tests pass on current main (#62); final suite evidence reconciled 09-11-2026.

Field	Content
TC ID	TC-SHP-02
Title	Stochastic Delay Modeling, Inbound Physical Receiving, and Delay Detection
Requirement(s)	SCM-F-022, SCM-F-023
Type	Functional
Objective	Verify that supplier reliability governs delay probabilities, that shipments are received into inventory only on or after their arrival day via `Inventory.add()`, that `delayed()` accurately detects both in-transit late and delivered late shipments, and that an arrival exceeding warehouse capacity stays in transit without aborting other arrivals.
Preconditions	An initialized `Inventory` instance and an instantiated `ShipmentManager`.
Test data	Suppliers `SUP-01` (reliability=1.0, lead_time=2), `SUP-SLOW` (reliability=0.0, lead_time=2); destination `WH-01`; item `SKU-01`; quantities 40, 75, 100; explicit delays 0 and 2.
Steps	1. Dispatch with `delay=0` on day 1 (arrival day 3) and call `process_arrivals(2)` and `process_arrivals(3)`. Verify stock and status.2. Dispatch with `delay=2` on day 1 (expected day 3, arrival day 5). Call `process_arrivals(3)` and verify shipment is not delivered early.3. Inspect `days_late()` across days 1 to 5, querying `ShipmentManager.delayed()`.4. Dispatch 20 shipments from `SUP-01` (`reliability=1.0`) across distinct PRNG seeds without explicit delay arguments.5. Dispatch from `SUP-SLOW` (`reliability=0.0`) and verify delay values.6. Dispatch a shipment that exceeds warehouse capacity and another that fits on the same day; call `process_arrivals`. Verify the oversized shipment remains `IN_TRANSIT`, is recorded in

Field	Content
	`last_blocked`, the fitting shipment is delivered, and freeing warehouse space allows the blocked shipment to deliver on the next call.
Expected result	On-time shipments deliver on expected day, incrementing inventory and setting status `DELIVERED`; delayed shipments do not arrive early; `delayed()` flags in-transit late shipments on day 4 and delivered late shipments on day 5; reliability 1.0 never delays; reliability 0.0 experiences delays in {1, 2, 3}. An arrival that would exceed warehouse capacity remains `IN_TRANSIT` with stock unchanged, is recorded in `last_blocked`, shows as `BLOCKED` in the day log, does not abort other arrivals, and is delivered once space frees up.
Pass/fail criteria	**Pass** if stock is updated solely via `inventory.add()`, arrival timing is strictly respected, all delay calculations evaluate correctly, and overflowing arrivals stay in transit without blocking other deliveries; **Fail** if premature delivery occurs, inventory is corrupted, or delays are miscalculated.
Automated by	`tests/test_shipments.py::test_on_time_arrival_adds_stocktests/test_shipments.py::test_delayed_shipment_not_delivered_earlytests/test_shipments.py::test_delayed_detection_both_in_transit_late_and_delivered_latetests/test_shipments.py::test_reliability_one_means_never_delayedtests/test_shipments.py::test_probabilistic_delay_with_zero_reliabilitytests/test_shipments.py::test_capacity_breach_does_not_abort_other_arrivalstests/test_simulator.py::test_blocked_arrival_is_logged_and_delivered_once_space_frees`
Actual result	Pass. Final automated suite: 100 tests passed, including blocked-arrival handling. See final unit-test report.
Status	Passed

Updated 08-11-2026: documented non-aborting arrivals when exceeding warehouse capacity (#70) (#62); final suite evidence reconciled 09-11-2026.

TC-SIM-01: Daily Step Order and Automatic Replenishment

Field	Content
TC ID	TC-SIM-01
Title	The daily step advances time, delivers shipments, fills existing back-orders before new demand, and trims automatic replenishment to warehouse capacity
Requirement(s)	SCM-F-023, SCM-F-041, SCM-F-042
Type	Integration
Preconditions	1. Python 3.11 virtual environment with `pip`

Field	Content
	`install -r requirements.txt` completed. 2. Repository at commit `b3702ef` (or a later green `main`). 3. Unless stated otherwise, each step uses a new scenario: `Simulation(seed=1)` with supplier SUP (lead time 2 days, reliability 1.0, so never delayed), warehouse WH with capacity 1000, and item (WH, SKU) with policy `StockPolicy(reorder_point, reorder_quantity, 500)` supplied by SUP.
Test data	Made-up IDs as used in the test code: supplier SUP, warehouse WH, SKU SKU, customers C1, C2. Defaults: initial stock 50, reorder point 20, reorder quantity 100. Step-specific values are given in each step.
Objective	Prove that `Simulation.step()` processes arrivals, retries existing back-orders before today's new demand, replenishes stock, and retains the daily report in the required order; automatic replenishment is capacity-safe and occurs at most once per item while a shipment is in transit.
Steps	1. **Day counter:** with stock 50, read `day`; call `step()`; then `run(3)`; read `day` and the `day` of every report in `history`. 2. **Arrival after lead time:** with stock 100, `step()` (day 1); dispatch 40 units from SUP on day 1; `step()` (day 2) and read the level; `step()` (day 3) and read the shipment status, `actual_day` and level. 3. **Back-order priority:** with stock 30 and no automatic replenishment, schedule C1 for 60 units on day 1 and step; schedule a day-2 shipment for 50 units and C2 for 40 units; step on day 2; read C1 and C2 status, and stock. 4. **Replenishment fulfilment:** with stock 30, schedule C1 for 60 units on day 1 and step; schedule C2 for 15 units on day 2; step; step on day 3; step on day 4; read ORD-0001 and stock after each step. 5. **Exactly one reorder:** with stock 10, `run(1)` three times, reading `shipments.in_transit()` and `logs[d].reorders`. 6. **Capacity trimming:** with warehouse capacity 50, policy `StockPolicy(20, 100, 50)` and stock 10, `run(3)`.
Expected result	1. `day` is 0 before stepping; after `step()` it is 1 and the report has `"day": 1`; after `run(3)` it is 4 and the history days are `[1, 2, 3, 4]`. 2. The shipment's `expected_day` is 3; on day 2 the level is still 100; on day 3 the status is DELIVERED, `actual_day == 3` and the level is 140. 3. Day 1: ORD-0001 is BACKORDERED. Day 2: the arriving 50 units are used to fill existing ORD-0001 first; the 40-unit C2 order remains BACKORDERED; stock is 20. 4. Day 1: ORD-0001 is BACKORDERED; day 2: the 15-unit order is fulfilled and one replenishment is in transit; day 3: ORD-0001 remains back-ordered; day 4: stock arrives first, so ORD-0001 is FULFILLED with `fulfilled_day == 4` and stock is 55. 5. Day 1: exactly 1 shipment is in transit and 1 reorder is

Field	Content
	logged; day 2: still 1 in transit and no new reorder; day 3: it arrives, leaving level 110 and 1 total reorder. 6. The reorder is trimmed to fit capacity, arrival does not raise `CapacityExceededError`, and the final level is 50.
Pass/fail criteria	**Pass** if every expected value is observed and all pytest tests listed below pass. **Fail** if the day counter or history is wrong, a shipment arrives early or late, an existing back-order is not prioritised over new demand, a back-order is not filled on the arrival day, a duplicate reorder is dispatched while one is in transit, or replenishment exceeds warehouse capacity.
Automated by	`tests/test_simulator.py::test_step_advances_day_counter`, `tests/test_simulator.py::test_shipment_arrives_after_lead_time_and_increases_stock`, `tests/test_simulator.py::test_large_order_is_backordered_then_fulfilled_after_replenishment`, `tests/test_simulator.py::test_low_stock_triggers_exactly_one_reorder_while_in_transit`, `tests/test_simulator.py::test_reorder_is_trimmed_to_fit_warehouse_capacity`, `tests/test_simulator.py::test_older_backorder_is_served_before_new_order_on_same_day`
Actual result	Pass. Final automated suite: 100 tests passed. See final unit-test report.
Status	Passed

Revision 09-11-2026: Added SCM-F-023 for verified shipment-arrival behavior; mapped CLI run-summary display to TC-SIM-02 / SCM-F-034 and CSV export to TC-SIM-02 / SCM-F-044, and reconciled automated evidence to the final unit-test report.

Field	Content
TC ID	TC-SIM-02
Title	System-Level Scenario Validation, Multi-Issue Detection, and CLI Execution
Requirement(s)	SCM-F-034, SCM-F-040, SCM-F-043, SCM-F-044, SCM-NF-005
Type	System
Objective	Perform end-to-end system verification on the Simulation and CLI modules to confirm that scenario scheduling rejects past days and invalid parameters, that the 10-day demo scenario detects all four supply chain issues (delays, stockouts, back-orders, and excess), and that the CLI runs successfully as `scm-sim` with `--summary`, `--csv`, and clean `--help` documentation.
Preconditions	Complete integrated package `scm_sim` installed with

Field	Content
	`pip install -e .` in a Python 3.11 environment, so the `scm-sim` command is available on PATH.
Test data	10-day standard demo scenario (`build_demo(seed=42)`): 2 suppliers (`SUP-RELIABLE`, `SUP-FLAKY`), 2 warehouses (`WH-NORTH`, `WH-SOUTH`), 3 SKUs (`WIDGET`, `GADGET`, `GIZMO`), 14 customer orders. Options `--summary` and `--csv <path>`.
Steps	1. Attempt scheduling an order for day 0 or past days (raises `ValueError`) and an unknown warehouse (raises `KeyError`).2. Execute `build_demo(seed=42).run(10)` and examine reports across all 10 simulation days.3. Verify that the simulation identifies delayed shipments, warehouse stockouts, back-ordered customer orders, and excess inventory.4. Invoke the CLI entry point `scm_sim.__main__.main(["--days", "10", "--seed", "42"])` and verify output and exit code 0.5. Execute Python sub-process module invocation `python -m scm_sim --days 1 --seed 42` and verify return code 0.6. Run the installed console command `scm-sim --days 10 --seed 42` as a subprocess.7. Run `scm-sim --days 3 --summary` and verify the `=== Run Summary ===` block.8. Run `scm-sim --days 3 --csv <path>` and verify the generated CSV structure.9. Run `scm-sim --days 0` and verify usage error.10. Run `scm-sim --help` and verify options documentation.
Expected result	`schedule_order` rejects non-future days and unknown warehouses; the 10-day demo scenario successfully triggers and reports delayed shipments, stockouts, back-ordered orders, and excess inventory; CLI executes cleanly with exit code 0; installed `scm-sim` runs; `--summary` prints run metrics; `--csv` writes header and daily rows; `--days 0` exits with code 2 and usage error; `--help` documentation displays all available options (`--days`, `--seed`, `--summary`, `--csv`).
Pass/fail criteria	**Pass** if all system features execute without failure, all four issue types are detected in reports, and CLI execution with `--summary` and `--csv` succeeds with exit code 0; **Fail** if exceptions are unhandled, issues go undetected, or CLI execution crashes.
Automated by	`tests/test_simulator.py::test_schedule_order_validates_inputtests/test_simulator.py::test_demo_detects_delay_stockout_backorder_and_excesstests/test_simulator.py::test_cli_main_runstests/test_simulator.py::test_cli_main_summary_flagtests/test_simulator.py::test_metrics_rows_has_one_flat_row_per_daytests/test_simulator.py::test_cli_csv_exporttests/test_simulator.py::test_cli_csv`

Field	Content
	`_and_summary_flagstests/test_simulator.py::test_cli_rejects_zero_daystests/test_simulator.py::test_cli_runs_as_moduletests/test_packaging.py::test_installed_scm_sim_command_runstests/test_packaging.py::test_pyproject_declares_scm_sim_console_scripttests/test_packaging.py::test_package_has_no_runtime_dependencies`
Actual result	Pass. Final automated suite: 100 tests passed, including package-install and CLI summary/CSV checks. See final unit-test report.
Status	Passed

Updated 08-11-2026: updated CLI steps and automated tests for `scm-sim`, `--summary` (#79) and `--csv` (#54) (#62); mapped `--summary` to SCM-F-034 and CSV export to SCM-F-044.